

[VALIDAR COM O AUTOR — NOME DA INSTITUIÇÃO DE ENSINO]

[VALIDAR COM O AUTOR — CAMPUS OU UNIDADE]

[VALIDAR COM O AUTOR — CURSO DE GRADUAÇÃO EM ENGENHARIA]

WILLIAM

**DESENVOLVIMENTO DE SOFTWARE ORIENTADO POR
ESPECIFICAÇÕES COM AGENTES DE INTELIGÊNCIA ARTIFICIAL:
UM ESTUDO DE CASO APLICADO A UM JOGO DIGITAL EM
ARTEFATO ÚNICO**

[VALIDAR COM O AUTOR — cidade]

WILLIAM

R ESPECIFICAÇÕES COM AGENTES DE INTELIGÊNCIA ARTIFICIAL: UM ESTUDO DE CAS

Trabalho de Conclusão de Curso apresentado ao curso de [VALIDAR COM O AUTOR — curso de graduação em Engenharia] da [VALIDAR COM O AUTOR — nome da instituição de ensino] como requisito parcial para a obtenção do título de Engenheiro

Orientador: [VALIDAR COM O AUTOR — Prof. Dr. Nome do Orientador]

Coorientador: [VALIDAR COM O AUTOR — ou declarar ausência]

Aprovado em: ____/____/____

[VALIDAR COM O AUTOR — Prof. Dr. Nome do Orientador]
Presidente da banca — [VALIDAR COM O AUTOR — nome da instituição de ensino]

[VALIDAR COM O AUTOR — Membro externo/interno 1]
[VALIDAR COM O AUTOR — titulação e instituição]

[VALIDAR COM O AUTOR — Membro externo/interno 2]
[VALIDAR COM O AUTOR — titulação e instituição]

RESUMO

Agentes de inteligência artificial generativa produzem código com alta velocidade, mas a ausência de um processo que force especificação prévia, critérios de aceitação verificáveis e registro de evidências compromete a auditabilidade e a manutenção do que foi produzido. Este trabalho define, aplica e avalia um processo de desenvolvimento orientado por especificações (*spec-driven development* — SDD) executado por agentes de inteligência artificial, organizado em quinze agentes especializados, quatro conjuntos de instruções permanentes e cinco habilidades de domínio, com fluxo adaptativo ao risco e seis portões de verificação. O processo foi aplicado à construção de um jogo digital completo entregue como artefato único de página web, especificado com quinze requisitos funcionais, nove requisitos não funcionais e doze critérios de aceitação. A verificação foi empírica, combinando inspeção estática do artefato com automação de navegador para teclado, apontador e leitura de pixels da área de desenho. Os doze critérios de aceitação foram satisfeitos, incluindo o bloqueio de inversão de trajetória, o sorteio de alvos fora do corpo do jogador e a persistência do recorde no armazenamento local. Mediu-se ainda o intervalo entre passos de simulação em duas faixas de pontuação: 133 ms para pontuação inferior a quarenta pontos e 105 ms para pontuação igual ou superior a oitenta pontos, sobre 184 movimentos amostrados, confirmando o aumento de velocidade previsto na especificação. O trabalho identifica como limitações a ausência de grupo de controle, a extensão reduzida da amostra temporal, a carência de critérios de aceitação para nove requisitos não funcionais e a não validação dos controles de toque em dispositivo físico. Conclui-se que o processo proposto torna cada entrega rastreável a um requisito e sustentada por evidência reproduzível, ao custo de maior disciplina documental.

Palavras-chave: desenvolvimento orientado por especificações; agentes de inteligência artificial; engenharia de requisitos; verificação e validação; rastreabilidade; estudo de caso.

Resumo

Generative artificial intelligence agents produce code at high speed, yet without a process that enforces prior specification, verifiable acceptance criteria and recorded evidence, the outcome is hard to audit and to maintain. This work defines, applies and evaluates a spec-driven development (SDD) process executed by artificial intelligence agents, organised into fifteen specialised agents, four permanent instruction sets and five domain skills, with a risk-adaptive flow and six verification gates. The process was applied to building a complete digital game delivered as a single web page artefact, specified with fifteen functional requirements, nine non-functional requirements and twelve acceptance criteria. Verification was empirical, combining static inspection of the artefact with browser automation for keyboard, pointer and pixel reading of the drawing surface. All twelve acceptance criteria passed, including trajectory reversal blocking, target spawning outside the player's body and high-score persistence in local storage. The interval between simulation steps was measured across two score bands: 133ms below forty points and 105ms at or above eighty points, over 184 sampled movements, confirming the speed increase defined in the specification. The work identifies as limitations the absence of a control group, the small temporal sample, the lack of acceptance criteria for nine non-functional requirements and the unvalidated touch controls on physical devices. It concludes that the proposed process makes every deliverable traceable to a requirement and supported by reproducible evidence, at the cost of greater documentation discipline.

Keywords: spec-driven development; artificial intelligence agents; requirements engineering; verification and validation; traceability; case study.

LISTA DE ILUSTRAÇÕES

Figura 1 – Camadas de revisão aplicadas à produção da monografia	24
Figura 2 – Camadas do framework SDD agêntico e responsabilidade de cada uma	35
Figura 3 – Fluxo agêntico adaptativo: classificação por complexidade e risco	39
Figura 4 – Pipeline SDD com os seis portões de verificação	40
Figura 5 – Cadeia de rastreabilidade do requisito à evidência	42
Figura 6 – Estrutura do artefato único e suas treze seções de código	49
Figura 7 – Máquina de estados da partida e transições	50
Figura 8 – Laço de animação com acumulador de tempo e execução de passos	51

Figura 9 – Algoritmo de um passo de simulação, com destaque para a ordem de ADR-004	53
Figura 10 – Sequência de tratamento de uma entrada direcional	55
Figura 11 – Interface implementada, no estado de fim de jogo com placar preenchido	59
Figura 12 – Curva do modelo de velocidade da especificação e os dois valores medidos	64

LISTA DE QUADROS

Figura 1.1 – Objetivos específicos e critério de sucesso	16
Figura 3.1 – Comparação entre abordagens correlatas segundo critérios de controle de processo	27
Figura 4.1 – Requisitos declarados na solicitação original do produto	29
Figura 4.2 – Etapas do procedimento e artefatos produzidos	29
Figura 4.3 – Ambiente experimental registrado	30
Figura 4.4 – Ameaças à validade identificadas e mitigações	33
Figura 5.1 – Lacunas identificadas no material de origem e tratamento no framework	35
Figura 5.2 – Agentes do framework, por grupo funcional	37
Figura 5.3 – Instruções permanentes do framework	38
Figura 5.4 – Trilhas do fluxo adaptativo e critério de elevação	40
Figura 5.5 – Portões de verificação do framework	40
Figura 6.1 – Requisitos funcionais do sistema-alvo	44
Figura 6.2 – Requisitos não funcionais do sistema-alvo	45
Figura 6.3 – Decisões arquiteturais do sistema-alvo	46
Figura 6.4 – Critérios de aceitação do sistema-alvo	47
Figura 7.1 – Trecho do laço de animação: acúmulo de tempo e execução de passos (excertado)	52
Figura 7.2 – Trecho do passo de simulação: cálculo da cabeça, colisão e remoção da cauda (excertado)	54
Figura 7.3 – Trecho do enfileiramento de direção com bloqueio de inversão (excertado)	54
Figura 7.4 – Trecho do sorteio de célula livre para o alvo (excertado)	55
Figura 7.5 – Trecho do ajuste da superfície de desenho à densidade da tela (excertado)	57
Figura 10.1 – Grau de cumprimento dos objetivos específicos	73

LISTA DE TABELAS

Tabela 1 – Mapeamento dos requisitos não funcionais do alvo nas características de qualidade da ISO/IEC 25010	21
Tabela 2 – Métricas operacionais, coleta e limitação	31
Tabela 3 – Constantes do alvo, conforme especificação e implementação	46
Tabela 4 – Grandezas temporais do laço de animação	52
Tabela 5 – Verificação das restrições na implementação	58
Tabela 6 – Resultado da verificação por critério de aceitação	61
Tabela 7 – Cobertura de rastreabilidade por classe de requisito	61
Tabela 8 – Intervalo médio entre passos medido e previsto, por faixa de pontuação	63
Tabela 9 – Itens não verificados e razão	65
Tabela 10 – Matriz de rastreabilidade requisito–critério–tarefa–código–teste–evidência	79
Tabela 11 – Lista de verificação de encerramento aplicada ao estudo de caso	81

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ADR	<i>Architecture Decision Record</i> (registro de decisão arquitetural)
API	<i>Application Programming Interface</i>
AP	<i>Access Point</i> (ponto de acesso sem fio)
CA	Critério de Aceitação
CSS	<i>Cascading Style Sheets</i>
DPR	<i>Device Pixel Ratio</i> (razão de pixels do dispositivo)
ECMA	<i>European Computer Manufacturers Association</i>
FR	Requisito Funcional
FSM	<i>Finite State Machine</i> (máquina de estados finitos)
HTML	<i>HyperText Markup Language</i>
IA	Inteligência Artificial
ISO	<i>International Organization for Standardization</i>
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i> (modelo de linguagem de grande porte)
NBR	Norma Brasileira
NFR	Requisito Não Funcional
RAM	<i>Random Access Memory</i>
SDD	<i>Spec-Driven Development</i>
SQuaRE	<i>Software product Quality Requirements and Evaluation</i>
SRP	<i>Single Responsibility Principle</i>
TCC	Trabalho de Conclusão de Curso
UI	<i>User Interface</i> (interface do usuário)

V&V	Verificação e Validação
W3C	<i>World Wide Web Consortium</i>
WCAG	<i>Web Content Accessibility Guidelines</i>
WHATWG	<i>Web Hypertext Application Technology Working Group</i>

LISTA DE SÍMBOLOS

A	Conjunto de critérios de aceitação verificados
C	Custo agregado de verificação
e	Número de itens consumidos na partida
$E[T]$	Intervalo médio entre passos de simulação
m	Número de amostras de movimento
s	Pontuação acumulada na partida
$t_{\max}$	Intervalo máximo entre passos de simulação
$t_{\min}$	Intervalo mínimo entre passos de simulação
t_0	Intervalo inicial entre passos de simulação
Δ	Passo de redução do intervalo por item consumido
σ	Desvio padrão amostral

SUMÁRIO

Lista de ilustrações	3
Lista de quadros	5
Lista de tabelas	6
1 INTRODUÇÃO	14
1.1 Contextualização	14
1.2 Problema de engenharia	14
1.3 Justificativa	15
1.4 Hipótese	15
1.5 Objetivos	16
1.5.1 Objetivo geral	16
1.5.2 Objetivos específicos	16
1.6 Delimitação do escopo	16
1.7 Visão resumida do método	17
1.8 Organização do trabalho	17
2 FUNDAMENTAÇÃO TEÓRICA	18
2.1 Engenharia de software e ciclo de vida	18
2.2 Especificação de requisitos	18
2.3 Verificação e validação	19
2.4 Testes e critérios de aceitação	19
2.5 Qualidade de produto de software	20
2.6 Arquitetura e decisões de projeto	20
2.7 Modelos de linguagem de grande porte	21
2.8 Agentes de inteligência artificial	22
2.9 Sistemas multiagente	22
2.10 Contexto, custo e disciplina documental	23
2.11 O domínio de aplicação	23
2.12 Síntese e lacuna identificada	24
3 TRABALHOS RELACIONADOS	25
3.1 Processos orientados por especificação	25
3.2 Agentes de linguagem no desenvolvimento de software	25
3.3 Padrões de raciocínio e autocorreção	26
3.4 Comparação	26

3.5	Posicionamento e contribuição pretendida	26
4	MATERIAIS E MÉTODOS	28
4.1	Natureza e abordagem da pesquisa	28
4.2	Unidade de análise e contexto	28
4.3	Procedimento	28
4.4	Instrumentos	29
4.5	Ambiente experimental	30
4.6	Variáveis	30
4.7	Métricas	31
4.8	Tratamento dos dados	32
4.9	Ética e integridade	32
4.10	Ameaças à validade	32
5	O FRAMEWORK SDD AGÊNTICO	34
5.1	Princípios de projeto	34
5.2	Motivação: lacunas do material de origem	34
5.3	Arquitetura em camadas	35
5.4	Agentes	36
5.5	Instruções permanentes	36
5.6	Habilidades de domínio	37
5.7	Fluxo adaptativo	38
5.8	Portões de verificação	38
5.9	Definição de concluído	39
5.10	Protocolo de contexto	41
5.11	Condições de parada	41
5.12	Matriz de rastreabilidade	42
5.13	Síntese do capítulo	42
6	ESPECIFICAÇÃO DO SISTEMA-ALVO	43
6.1	Contexto de especificação	43
6.2	Requisitos funcionais	43
6.3	Requisitos não funcionais	44
6.4	Restrições e diretrizes	44
6.5	Decisões arquiteturais	45
6.6	Contratos de dados	45
6.7	Regra de evolução da velocidade	45
6.8	CrITÉRIOS de aceitação	47
6.9	Casos de borda declarados	47
6.10	Síntese do capítulo	48

7	IMPLEMENTAÇÃO	49
7.1	Estrutura do artefato	49
7.2	Máquina de estados da partida	50
7.3	Laço de animação	50
7.4	Algoritmo de um passo de simulação	52
7.5	Fila de entrada e bloqueio de inversão	53
7.6	Sorteio do alvo	54
7.7	Persistência do recorde	56
7.8	Renderização	56
7.9	Acessibilidade e resposta a preferências	57
7.10	Verificação de restrições durante a implementação	58
7.11	Síntese do capítulo	58
8	EXPERIMENTOS E RESULTADOS	60
8.1	Protocolo de verificação	60
8.2	Resultados por critério de aceitação	60
8.3	Cobertura de rastreabilidade	61
8.4	Verificação de propriedades estruturais	62
8.5	Medição do intervalo entre passos	62
8.5.1	Procedimento	63
8.5.2	Resultado	63
8.6	Comportamentos observados sem critério correspondente	64
8.7	O que não foi verificado	65
8.8	Síntese do capítulo	65
9	DISCUSSÃO	66
9.1	Retomada do problema e da hipótese	66
9.2	A assimetria entre requisitos funcionais e não funcionais	66
9.2.1	A lacuna do portão de intenção	67
9.3	Valor dos casos de borda e das decisões arquiteturais	68
9.4	Custo do processo	68
9.5	Ameaças à validade revisitadas	69
9.6	Generalização	70
9.7	Comparação com a literatura	70
9.8	Implicações práticas	71
10	CONCLUSÃO	72
10.1	Resposta ao problema	72
10.2	Cumprimento dos objetivos	72
10.3	Contribuições	73

10.4	Limitações	73
10.5	Trabalhos futuros	74
10.6	Considerações finais	75

REFERÊNCIAS	76
-----------------------	----

APÊNDICES 78

	APÊNDICE A – MATRIZ DE RASTREABILIDADE COMPLETA .	79
A.1	Leitura da matriz	80
	APÊNDICE B – LISTA DE VERIFICAÇÃO DE ENCERRAMENTO	
	APLICADA	81
B.1	Leitura da lista	82

1 INTRODUÇÃO

Este capítulo apresenta o contexto do trabalho, o problema de engenharia investigado, a justificativa, a hipótese, os objetivos, a delimitação do escopo, uma visão resumida do método e a organização dos demais capítulos.

1.1 Contextualização

A produção de software assistida por modelos de linguagem de grande porte (*Large Language Models* — LLM) deixou de ser experimento de laboratório e passou a integrar rotinas reais de desenvolvimento. Modelos treinados em larga escala demonstram capacidade de generalização para tarefas não vistas durante o treinamento, fenômeno descrito como aprendizado em poucos exemplos (Brown et al. (2020)). A arquitetura de atenção proposta por Vaswani et al. (2017) viabilizou essa classe de modelos, e técnicas de encadeamento de raciocínio ampliaram a capacidade de resolução de problemas estruturados (Wei et al. (2022)).

A partir dessa base, surgiram abordagens que organizam o modelo como agente capaz de raciocinar e agir sobre um ambiente (Yao et al. (2023)), de refletir sobre os próprios erros (Shinn et al. (2023)) e de operar em conjunto com outros agentes, cada um com papel especializado (Hong et al. (2024); Qian et al. (2024); Wu et al. (2023)).

O ganho de velocidade é evidente; o ganho de qualidade é condicional. Um agente que recebe um pedido vago produz código plausível, mas plausibilidade não é correteude. Quando não existe especificação prévia, não há como decidir se o que foi entregue é o que foi pedido; quando não existem critérios de aceitação, não há como demonstrar que o pedido foi atendido; e quando não há registro de evidência, a conclusão depende de confiança, não de verificação. A engenharia de software já conhecia esse problema: a distinção entre verificação e validação e a necessidade de critérios objetivos foram formalizadas há décadas (Boehm (1984)), e a especificação de requisitos permanece sendo a atividade com maior impacto sobre o custo final do produto (Sommerville (2019); Pressman e Maxim (2016)).

1.2 Problema de engenharia

O problema investigado é:

Como estruturar um processo de desenvolvimento assistido por agentes de inteligência artificial de modo que cada entrega seja (i) rastreável a um requisito, (ii) verificável por um critério de aceitação e (iii) sustentada por evidência reproduzível?

O problema se decompõe em três dificuldades práticas. A primeira é de natureza processual: agentes tendem a responder ao pedido mais recente, e não ao contrato acumulado, de modo que sem um artefato de estado persistente o escopo se degrada ao longo da execução. A segunda é de natureza epistemológica: a saída de um agente é texto plausível, e plausibilidade não é evidência; é necessário distinguir o que foi afirmado do que foi demonstrado. A terceira é de natureza econômica: quanto maior o volume de contexto carregado, maior o custo e maior a probabilidade de o agente tratar informação obsoleta como atual.

1.3 Justificativa

A relevância do estudo se sustenta em três argumentos.

Argumento de risco. As práticas que reduzem risco de regressão em software — integração contínua, testes automatizados, releases frequentes e pequenas — estão associadas a melhor desempenho organizacional (Humble e Farley (2010); Forsgren, Humble e Kim (2018)). Um processo que aceita código gerado por agente sem passar pelos mesmos portões de qualidade importa risco para dentro do produto.

Argumento de auditabilidade. Em domínios regulados ou críticos, a pergunta relevante não é apenas se o software funciona, mas quem decidiu o quê, com base em qual requisito e com qual evidência. A rastreabilidade é um requisito de engenharia, não um subproduto (International Organization for Standardization (2018)).

Argumento de reprodutibilidade. Se duas execuções do mesmo pedido produzem resultados diferentes e não existe especificação, o resultado não é comparável nem acumulável. Sem artefatos congelados, o conhecimento produzido em uma sessão se perde na seguinte.

Além disso, a literatura sobre agentes LLM aplicados à engenharia de software concentra-se majoritariamente na geração de código (Hong et al. (2024); Qian et al. (2024)). O presente trabalho desloca o foco da geração para o *controle*: quais artefatos, portões e condições de parada tornam a geração auditável.

1.4 Hipótese

A hipótese de trabalho é:

Um processo que obriga a especificação prévia com requisitos identificados, critérios de aceitação testáveis e evidência registrada produz entregas verificáveis por terceiros sem acesso ao histórico da conversa, ao custo de maior esforço documental.

A hipótese é falseável de duas maneiras. Primeiro, se algum critério de aceitação não puder ser verificado sem consultar o autor, a rastreabilidade proposta falhou. Segundo, se a evidência registrada não puder ser reproduzida a partir dos artefatos versionados, o processo não é reprodutível. Ambos os testes foram aplicados neste trabalho, e os resultados estão no Capítulo 8.

1.5 Objetivos

1.5.1 Objetivo geral

Definir, aplicar e avaliar um processo de desenvolvimento orientado por especificações (*spec-driven development* — SDD) executado por agentes de inteligência artificial, utilizando como estudo de caso a construção de um jogo digital completo entregue em artefato único.

1.5.2 Objetivos específicos

Quadro 1.1 – Objetivos específicos e critério de sucesso

ID	Objetivo específico	Critério de sucesso
OE-1	Projetar o framework de agentes, instruções e artefatos de controle.	Framework descrito e versionado no repositório.
OE-2	Especificar formalmente o sistema-alvo com requisitos e critérios de aceitação verificáveis.	Todo requisito com identificador único.
OE-3	Registrar as decisões arquiteturais do alvo com alternativas descartadas.	Cada decisão com justificativa e alternativa rejeitada.
OE-4	Implementar o alvo sob as restrições especificadas.	Artefato único, sem dependências externas.
OE-5	Verificar cada critério de aceitação com evidência empírica.	Um resultado observável por critério.
OE-6	Avaliar o processo, suas limitações e ameaças à validade.	Ameaças classificadas e dados ausentes marcados.

Fonte: elaborado pelo autor (2026).

1.6 Delimitação do escopo

Dentro do escopo: o processo de desenvolvimento, sua especificação, sua aplicação a um sistema-alvo e a verificação empírica do alvo.

Fora do escopo: o treinamento ou a avaliação comparativa de modelos de linguagem; a medição de custo financeiro por execução; a avaliação de usabilidade com usuários humanos; e a comparação experimental contra grupo de controle, que exigiria um segundo desenvolvimento paralelo do mesmo alvo.

Registra-se ainda uma delimitação factual relevante. O repositório onde o trabalho foi conduzido contém a descrição de um sistema embarcado de controle térmico, sem código, medição ou resultado correspondente. Como não há evidência empírica desse sistema, ele não é objeto deste estudo; o estudo de caso é o produto efetivamente implementado e verificado. Essa inconsistência documental está registrada em `MONOGRAFIA_PENDENCIAS.md` (P-26).

1.7 Visão resumida do método

O trabalho é um estudo de caso único, de natureza aplicada, com abordagem qualitativa predominante e componentes quantitativos pontuais (Yin (2018); Runeson e Höst (2009)). O procedimento teve quatro etapas: (i) projeto do framework de agentes a partir das lacunas identificadas em uma avaliação crítica prévia do material de origem; (ii) especificação do alvo com requisitos funcionais, não funcionais e critérios de aceitação; (iii) implementação sob restrições declaradas; e (iv) verificação empírica em navegador, com inspeção estática e automação. O detalhamento está no Capítulo 4.

1.8 Organização do trabalho

O Capítulo 2 apresenta a fundamentação teórica em engenharia de software e agentes de inteligência artificial. O Capítulo 3 compara trabalhos correlatos. O Capítulo 4 descreve materiais e métodos, incluindo ameaças à validade. O Capítulo 5 detalha o framework SDD agêntico proposto. O Capítulo 6 apresenta a especificação do sistema-alvo. O Capítulo 7 descreve a implementação e suas decisões arquiteturais. O Capítulo 8 reporta experimentos e resultados. O Capítulo 9 discute os achados e as limitações. O Capítulo 10 conclui e indica trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo estabelece os conceitos necessários para compreender o problema e a solução proposta. A primeira parte trata de engenharia de software — ciclo de vida, requisitos, verificação e validação, testes e qualidade. A segunda trata de agentes de inteligência artificial — modelos de linguagem, agentes autônomos e sistemas multiagente. A terceira trata do domínio de aplicação. A seção final sintetiza as duas áreas e identifica a lacuna que o trabalho aborda.

2.1 Engenharia de software e ciclo de vida

Engenharia de software é a aplicação de abordagem sistemática, disciplinada e quantificável ao desenvolvimento, operação e manutenção de software (Sommerville (2019)). A palavra decisiva é *sistemática*: a diferença entre programar e engenhar software está na existência de um processo que torne o resultado previsível e inspecionável.

Processos de desenvolvimento organizam-se tradicionalmente em fases — elicitação, análise, projeto, implementação, teste, implantação e manutenção — ainda que a execução real raramente seja estritamente sequencial (Pressman e Maxim (2016)). Modelos incrementais e iterativos reconhecem que requisitos mudam e que a retroalimentação é inevitável. O que não se altera entre modelos é a assimetria de custo: quanto mais tarde um defeito de requisito é descoberto, maior o custo de corrigi-lo. Essa assimetria pode ser formalizada como uma progressão geométrica sobre as fases do ciclo de vida:

$$C(f) = C_0 \cdot \alpha^{f-1}, \quad \alpha > 1 \quad (2.1)$$

em que $C(f)$ é o custo de correção quando o defeito é detectado na fase f , C_0 é o custo de correção na fase de requisitos e α é o fator de ampliação por fase. **O parâmetro α depende do domínio e não foi medido neste trabalho**; a equação serve para formalizar a direção do efeito, não para quantificá-lo. A implicação prática é direta: investir em especificação é investir em reduzir f .

2.2 Especificação de requisitos

A norma ISO/IEC/IEEE 29148 define os processos e os produtos da engenharia de requisitos, incluindo as características que tornam um requisito verificável (International Organization for Standardization (2018)). Um requisito bem formado é singular, não ambíguo, completo, consistente, factível, necessário e — sobretudo para os fins deste

trabalho — *verificável*: deve existir um procedimento finito e objetivo capaz de decidir se o requisito foi atendido.

A literatura separa requisitos em duas classes. Requisitos funcionais descrevem o que o sistema deve fazer. Requisitos não funcionais descrevem como o sistema deve se comportar em atributos de qualidade — desempenho, usabilidade, segurança, manutenibilidade (Sommerville (2019)). Requisitos não funcionais são notoriamente mais difíceis de verificar, porque frequentemente são enunciados como adjetivos. A prática recomendada é convertê-los em métricas: em vez de “deve ser rápido”, “o intervalo entre passos de simulação não deve exceder $t_{\max}$ ”.

Uma especificação útil também precisa declarar suas *restrições* — decisões já tomadas que o projeto não pode contradizer — e seus *casos de borda*, isto é, as entradas ou estados limítrofes que costumam revelar defeitos. Este trabalho adota a convenção de nomear cada elemento com identificador estável (FR-XXX, NFR-XXX, CA-XXX), prática que viabiliza a rastreabilidade automática entre artefatos.

2.3 Verificação e validação

Verificação e validação são atividades distintas, distinção formalizada por Boehm (1984). A verificação responde à pergunta “o produto foi construído corretamente?”, isto é, se ele está em conformidade com a especificação. A validação responde à pergunta “foi construído o produto correto?”, isto é, se ele satisfaz a necessidade que motivou o projeto.

A distinção tem consequência prática imediata para este trabalho. A verificação de doze critérios de aceitação demonstra conformidade com a especificação; ela *não* demonstra que o produto é útil, agradável ou que o público-alvo o adotaria. Afirmações sobre adequação ao uso exigiriam validação com usuários, que está fora do escopo declarado na Seção 1.6.

Boehm (1984) argumenta ainda que a verificação deve incidir preferencialmente sobre os artefatos iniciais — requisitos e projeto — porque são eles que determinam a maior parte do custo. Verificar apenas código é verificar tarde.

2.4 Testes e critérios de aceitação

Diversas estratégias de teste coexistem. Myers, Sandler e Badgett (2011) enfatiza que o objetivo do teste é *encontrar defeitos*, não confirmar que o sistema funciona, e que um caso de teste que não pode falhar não tem valor. Daí o princípio de projetar casos a partir de condições limítrofes e de entradas inválidas.

Duas práticas são diretamente relevantes para o processo proposto. A primeira é o desenvolvimento orientado por testes, que inverte a ordem habitual: o teste que expressa o comportamento desejado é escrito antes da implementação (Beck (2002)). A segunda é a especificação por exemplo, que substitui descrições abstratas por exemplos concretos e executáveis do comportamento esperado (Adzic (2011)). Ambas compartilham a mesma tese: o critério de aceitação precisa existir antes da implementação, sob pena de se tornar uma racionalização posterior do que foi produzido.

Neste trabalho, o critério de aceitação é o elo entre requisito e evidência. A cobertura de verificação é definida como:

$$A = \frac{|\{CA_i \mid \text{resultado}(CA_i) = \text{PASS}\}|}{|\{CA_i\}|} \quad (2.2)$$

em que A é a fração de critérios de aceitação satisfeitos. A equação mede conformidade com a especificação — verificação, no sentido de Boehm (1984) — e não adequação ao uso.

2.5 Qualidade de produto de software

A ISO/IEC 25010 organiza a qualidade de produto em características como adequação funcional, eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade (International Organization for Standardization (2011)). O valor prático desse modelo, para este trabalho, é oferecer vocabulário para classificar requisitos não funcionais que, de outro modo, permaneceriam adjetivos soltos.

A Tabela 1 mapeia os requisitos não funcionais do alvo nas características da norma. O mapeamento revela uma concentração: oito dos nove requisitos não funcionais distribuem-se em apenas três características, o que indica que a especificação priorizou fortemente portabilidade e adequação de interface, em detrimento de eficiência de desempenho e confiabilidade.

A ausência de requisito não funcional dedicado a eficiência de desempenho é uma lacuna da especificação, não do produto: o alvo executa um laço contínuo de renderização e, sem meta declarada de consumo ou de latência, não há critério objetivo para reprová-lo nesse aspecto. Essa constatação reaparece na discussão (Capítulo 9).

2.6 Arquitetura e decisões de projeto

Arquitetura de software é o conjunto de decisões estruturais cujo custo de mudança é alto (Bass, Clements e Kazman (2021)). Documentar decisões arquiteturais — o problema enfrentado, a alternativa escolhida, as alternativas descartadas e a justificativa — preserva

Tabela 1 – Mapeamento dos requisitos não funcionais do alvo nas características de qualidade da ISO/IEC 25010

Característica (ISO/IEC 25010)	Requisitos não funcionais do alvo
Adequação funcional	NFR-008 (ausência de erros no carregamento)
Eficiência de desempenho	— (nenhum requisito não funcional dedicado)
Compatibilidade	NFR-001 (sem dependências externas)
Usabilidade	NFR-002 (layout centralizado), NFR-003 (tema escuro), NFR-007 (entrada não bloqueante), NFR-009 (movimento reduzido)
Confiabilidade	— (apenas tolerância a falha de armazenamento, tratada em FR-010)
Segurança	— (fora do escopo: aplicação local, sem rede nem dados pessoais)
Manutenibilidade	NFR-005 (código comentado e organizado)
Portabilidade	NFR-004 (responsividade), NFR-006 (nitidez em telas de alta densidade)

Fonte: elaborado pelo autor (2026) a partir de International Organization for Standardization (2011) e da especificação do alvo.

o raciocínio que deu origem ao projeto. Sem esse registro, o código permanece como única fonte, e o código diz *o que* foi feito, não *por que* nem *em vez do quê*.

Padrões de projeto oferecem soluções nomeadas para problemas recorrentes (Gamma et al. (1994)), e a separação de responsabilidades é critério central de qualidade estrutural. Neste trabalho, dois padrões são pertinentes: a separação entre atualização de estado e apresentação, que isola a lógica de simulação da lógica de desenho; e o uso de um objeto único de configuração, que centraliza constantes e evita a duplicação de valores mágicos.

A integração contínua e a entrega contínua fornecem o contexto organizacional em que portões de verificação automatizados se inserem (Humble e Farley (2010); Forsgren, Humble e Kim (2018)). O framework proposto no Capítulo 5 é uma adaptação desses portões para um fluxo executado por agentes.

2.7 Modelos de linguagem de grande porte

A arquitetura *Transformer*, baseada em mecanismos de atenção, substituiu as recorrências anteriores por processamento paralelo de dependências de longo alcance (Vaswani et al. (2017)). Sobre essa base, modelos com centenas de bilhões de parâmetros demonstraram capacidade de executar tarefas a partir de poucos exemplos fornecidos no próprio texto de entrada, sem re-treinamento (Brown et al. (2020)).

A capacidade que interessa a este trabalho não é a fluência, e sim o *raciocínio em etapas*. Wei et al. (2022) mostraram que solicitar ao modelo que explicita o raciocínio

intermediário melhora o desempenho em tarefas que exigem múltiplos passos. Isso importa porque verificação, análise de requisito e decomposição de tarefa são, todas, atividades de múltiplos passos.

Duas limitações precisam ser declaradas, porque afetam a confiabilidade do processo. A primeira é a geração de conteúdo plausível e incorreto, produzido com o mesmo grau de confiança do conteúdo correto; é precisamente esse fenômeno que torna indispensável exigir evidência independente da afirmação. A segunda é a dependência do contexto: o comportamento do modelo é função do que está na janela de entrada, de modo que informação ausente, obsoleta ou contraditória produz saída degradada.

2.8 Agentes de inteligência artificial

Um agente é um sistema que percebe um ambiente e age sobre ele (Russell e Norvig (2021)). A transposição dessa definição para modelos de linguagem produziu agentes capazes de intercalar raciocínio e ação, alternando etapas de deliberação com a execução de ferramentas e a observação do resultado (Yao et al. (2023)). A capacidade de refletir sobre o próprio erro, armazenando a avaliação crítica e reutilizando-a em tentativas seguintes, melhora o desempenho em tarefas iterativas (Shinn et al. (2023)).

Agentes introduzem dois problemas ausentes em modelos usados como meros geradores de texto. O primeiro é a *deriva de objetivo*: um agente que pode escolher o próximo passo pode escolher um passo que não serve ao objetivo original. O segundo é a *opacidade*: a sequência de decisões que levou ao artefato final pode não ser reconstruível. Ambos são problemas de controle, e é para eles que o framework do Capítulo 5 se orienta.

2.9 Sistemas multiagente

Sistemas multiagente clássicos estudam coordenação, cooperação e negociação entre agentes autônomos (Wooldridge (2009)). Aplicada a modelos de linguagem, a ideia deu origem a frameworks que distribuem papéis especializados. MetaGPT organiza papéis análogos a uma equipe de software e representa os artefatos intermediários de forma estruturada (Hong et al. (2024)). ChatDev modela o desenvolvimento como diálogo entre agentes em papéis de projetista, programador e revisor (Qian et al. (2024)). AutoGen generaliza a conversação entre agentes configuráveis como primitiva de construção (Wu et al. (2023)).

Esses trabalhos tratam principalmente da *geração cooperativa*. A contribuição pretendida neste trabalho é complementar: trata-se de fixar os *artefatos de controle* — especificação, decisão, tarefa, evidência — e as condições sob as quais o fluxo deve parar por falta de informação. A comparação detalhada está no Capítulo 3.

2.10 Contexto, custo e disciplina documental

Como o comportamento do agente é função da janela de contexto, o volume de documentação carregada tem custo duplo: custo computacional e custo de ruído. Informação irrelevante não é neutra — ela compete por atenção com a informação relevante. Daí decorre um princípio de projeto adotado neste trabalho: documentos de controle devem ser curtos, e a leitura do repositório deve ser incremental e orientada à tarefa corrente, não exaustiva.

Em tensão com esse princípio está a exigência de rastreabilidade, que empurra na direção oposta, a de mais documentação. O framework proposto resolve a tensão por especialização: cada informação tem *um* documento dono, e os demais apontam para ele em vez de repeti-lo. Uma matriz de rastreabilidade existe para conectar, não para duplicar.

2.11 O domínio de aplicação

O estudo de caso é um jogo digital de movimentação em grade, entregue como página web única. Três características do domínio tornam-no adequado ao propósito.

Estado discreto e verificável. O jogo é uma máquina de estados finitos sobre uma grade discreta, com condições de término bem definidas (colisão com borda ou com o próprio corpo). Isso permite afirmar propriedades precisas e verificá-las por observação, sem depender de julgamento subjetivo.

Lógica de tempo real. O jogo exige um laço contínuo de atualização e desenho. A plataforma web oferece `requestAnimationFrame` para sincronizar o desenho com a taxa de atualização do dispositivo (Mozilla Foundation (2024c)), e o desacoplamento entre taxa de simulação e taxa de quadros é uma decisão de projeto com consequências mensuráveis (Capítulo 7).

Apresentação e persistência locais. A superfície de desenho vetorial bidimensional é adequada a grade discreta (Mozilla Foundation (2024a)), e o armazenamento local do navegador permite persistir estado entre sessões sem servidor (Mozilla Foundation (2024b)). A acessibilidade é orientada pelas diretrizes WCAG (World Wide Web Consortium (2023)), e a linguagem é regida pelo padrão ECMAScript (Ecma International (2023)) sobre a especificação viva de HTML (Web Hypertext Application Technology Working Group (2024)).

2.12 Síntese e lacuna identificada

A fundamentação apresentada sustenta três conclusões que orientam o restante do trabalho.

Primeira: a verificação depende de critérios definidos *antes* da implementação; critérios escritos depois são descrição, não verificação.

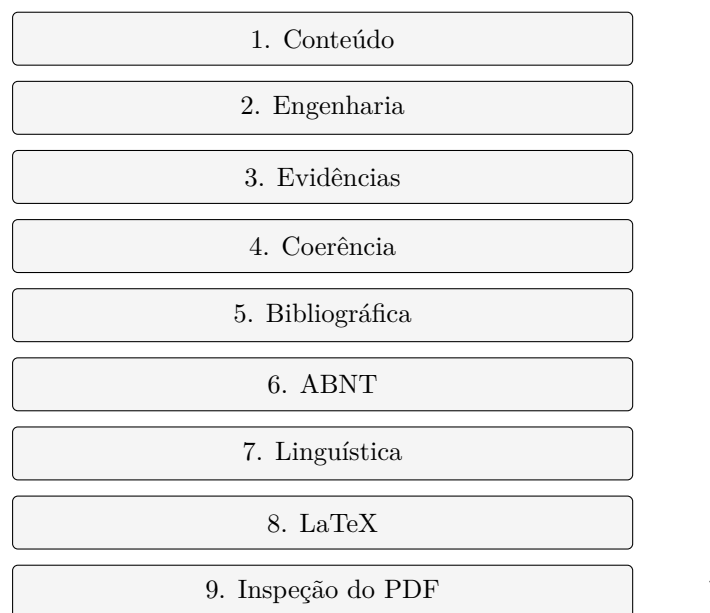
Segunda: requisitos não funcionais só são verificáveis quando convertidos em métricas; quando permanecem adjetivos, eles não entram em nenhum portão de aceitação, e a lacuna passa despercebida justamente porque não há teste que possa falhar.

Terceira: agentes de linguagem são úteis para gerar e para revisar, mas não são fonte de evidência. A evidência precisa vir de execução observável e reproduzível.

A lacuna que o trabalho aborda é a ausência, na literatura de agentes LLM para engenharia de software, de um processo que trate explicitamente os artefatos de controle e as condições de parada, com ênfase em verificabilidade medida. O Capítulo 3 sustenta essa afirmação por comparação.

A verificação, por sua vez, não é um evento único e sim um conjunto de camadas aplicadas em momentos distintos do trabalho. A Figura 1 representa essas camadas e registra quais foram efetivamente aplicadas na produção deste documento. A distinção entre camadas aplicadas e camadas não executadas é parte do resultado, não uma omissão.

Figura 1 – Camadas de revisão aplicadas à produção da monografia



Fonte: elaborado pelo autor (2026). Neste trabalho, as camadas 1 a 5 foram aplicadas durante a produção; a camada 6 foi parcialmente aplicada e depende do manual institucional; as camadas 8 e 9 **não foram executadas** por ausência de compilador LaTeX no ambiente — ver `MONOGRAFIA_STATUS.md`, seção 3.

3 TRABALHOS RELACIONADOS

Este capítulo posiciona o trabalho em relação a três linhas: processos orientados por especificação, agentes de linguagem aplicados ao desenvolvimento de software e verificação de software assistida por agentes. A comparação é predominantemente qualitativa; onde uma comparação quantitativa exigiria dados que não foram consultados na fonte original, isso é declarado.

3.1 Processos orientados por especificação

O desenvolvimento orientado por testes estabelece o teste como artefato de primeira classe, escrito antes da implementação (Beck (2002)). A especificação por exemplo leva a ideia adiante, substituindo descrição abstrata por exemplos concretos que funcionam simultaneamente como requisito, teste e documentação (Adzic (2011)). Ambos compartilham o mecanismo central adotado neste trabalho: o critério de aceitação precede a implementação.

A engenharia de requisitos normatizada fornece o vocabulário para tornar o critério verificável, incluindo atributos de qualidade de requisito (International Organization for Standardization (2018)). O modelo de qualidade de produto (International Organization for Standardization (2011)) fornece a taxonomia para classificar requisitos não funcionais.

O desenvolvimento de linguagens específicas de domínio ilustra o caso extremo da mesma tese: quando o domínio é bem compreendido, uma notação restrita reduz a ambiguidade e a possibilidade de erro (Mernik, Heering e Sloane (2005)). O paralelo com este trabalho está na função dos identificadores estáveis: ao nomear cada requisito, critério e tarefa, restringe-se o espaço de referências ambíguas.

3.2 Agentes de linguagem no desenvolvimento de software

Três trabalhos representativos organizam múltiplos agentes para produzir software. MetaGPT atribui papéis análogos aos de uma equipe e representa artefatos intermediários de forma estruturada, com o argumento de que representações intermediárias bem definidas reduzem a propagação de erro entre etapas (Hong et al. (2024)). ChatDev modela o desenvolvimento como conversação entre papéis de projetista, programador e revisor, com foco na comunicação entre agentes como mecanismo de correção (Qian et al. (2024)). AutoGen trata a conversação entre agentes configuráveis como primitiva de construção, permitindo compor fluxos com participação humana (Wu et al. (2023)).

Esses trabalhos demonstram que a decomposição em papéis melhora a qualidade do artefato gerado. Entretanto, o critério de sucesso predominante é a qualidade do código ou a taxa de conclusão da tarefa, e não a existência de uma cadeia auditável entre requisito, teste e evidência. Em particular, nenhum dos três define condições de parada por informação ausente como parte do protocolo: o agente pode continuar produzindo na ausência de dados essenciais.

3.3 Padrões de raciocínio e autocorreção

O intercalamento entre raciocínio e ação permite que o agente use o resultado de uma ação para decidir a próxima, corrigindo a trajetória durante a execução (Yao et al. (2023)). A reflexão sobre o próprio erro, com registro da avaliação crítica reutilizada em tentativas posteriores, melhora o desempenho em tarefas que admitem tentativa e erro (Shinn et al. (2023)). A explicitação do raciocínio intermediário melhora o desempenho em tarefas de múltiplos passos (Wei et al. (2022)).

Esses padrões são adotados no framework proposto, mas com uma restrição: a autocorreção é aplicada a artefatos internos (especificação, tarefas, código), nunca à evidência. A evidência de teste não é objeto de reflexão do agente; ela é resultado de execução. Essa separação é deliberada e responde ao problema de opacidade apontado na Seção 2.8.

3.4 Comparação

O Quadro 3.1 compara os trabalhos segundo cinco critérios derivados do problema de pesquisa.

3.5 Posicionamento e contribuição pretendida

A comparação do Quadro 3.1 evidencia que as abordagens existentes cobrem bem a geração cooperativa e os padrões de raciocínio, mas não combinam, em um único processo, quatro elementos: especificação prévia obrigatória, critério verificável com identificador estável, evidência de execução como única fonte de conclusão e parada explícita diante de informação ausente.

A contribuição pretendida não é um novo modelo de linguagem nem um novo algoritmo de coordenação. É um *protocolo de controle* aplicável sobre agentes existentes, com três propriedades verificáveis:

Quadro 3.1 – Comparação entre abordagens correlatas segundo critérios de controle de processo

Abordagem	<i>Especificação prévia</i>	<i>Múltiplos papéis</i>	<i>Critério verificável</i>	<i>Evidência de execução</i>	<i>Parada por dado ausente</i>
Beck (2002)	Parcial	Não	Sim	Sim	Não
Adzic (2011)	Sim	Não	Sim	Sim	Não
International Organization for Standardization (2018)	Sim	Não	Sim	Não	Não
Hong et al. (2024)	Parcial	Sim	Não	Não	Não
Qian et al. (2024)	Parcial	Sim	Não	Não	Não
Wu et al. (2023)	Não	Sim	Não	Parcial	Não
Yao et al. (2023)	Não	Não	Não	Sim	Não
Shinn et al. (2023)	Não	Não	Não	Sim	Não
Processo proposto	Sim	Sim	Sim	Sim	Sim

Fonte: elaborado pelo autor (2026). A classificação é qualitativa e baseada na leitura dos trabalhos; “parcial” indica que o critério existe de forma implícita ou incompleta. Confirmar cada classificação contra o texto original — ver P-16 a P-21.

1. **Rastreabilidade** — cada entrega é referenciável a um requisito. 2. **Falseabilidade** — cada requisito relevante tem um teste capaz de reprová-lo. 3. **Honestidade epistêmica** — ausência de dado é registrada como ausência, nunca suprida por estimativa.

A avaliação dessa contribuição é feita por estudo de caso, conforme o Capítulo 4, e não por experimento controlado. Essa escolha metodológica tem custo e benefício, discutidos na Seção 4.10.

4 MATERIAIS E MÉTODOS

Este capítulo descreve a natureza da pesquisa, o procedimento, os instrumentos, o ambiente experimental, as variáveis, as métricas, o tratamento dos dados e as ameaças à validade. O objetivo é permitir que outro pesquisador avalie criticamente o trabalho e reproduza o que foi executado.

4.1 Natureza e abordagem da pesquisa

A pesquisa é de natureza **aplicada**, com objetivo **exploratório e descritivo**, e abordagem **qualitativa predominante com componentes quantitativos pontuais**. O método é o **estudo de caso único**, adequado quando o fenômeno é contemporâneo, não manipulável pelo pesquisador e inseparável do contexto em que ocorre (Yin (2018)). As diretrizes para condução e relato de estudos de caso em engenharia de software (Runeson e Höst (2009)) orientaram a estrutura deste capítulo.

A escolha do estudo de caso em detrimento do experimento controlado é consequência da pergunta de pesquisa. A pergunta é “como estruturar um processo”, e processos são artefatos sócio-técnicos cuja validade depende de como são operados na prática. Um experimento controlado exigiria um grupo de controle submetido ao mesmo pedido sem o processo proposto — o que foi registrado como dado necessário e não obtido (P-14).

4.2 Unidade de análise e contexto

A unidade de análise é a **execução do processo** sobre um sistema-alvo. O sistema-alvo é um jogo digital de movimentação em grade, entregue como artefato único de página web, com os seguintes requisitos de produto declarados na solicitação original:

4.3 Procedimento

O procedimento teve quatro etapas encadeadas, cada uma produzindo artefatos consumidos pela seguinte. O pipeline completo está no Quadro 4.2.

A ordem das etapas não é negociável dentro do processo: a especificação precede a implementação e a verificação conclui. Essa precedência é a variável independente do estudo.

Quadro 4.1 – Requisitos declarados na solicitação original do produto

#	Requisito declarado na solicitação
R1	Arquivo único com CSS e JavaScript embutidos na mesma página.
R2	Ausência de dependências externas: apenas desenho vetorial, CSS e JavaScript nativos.
R3	Layout moderno, limpo, responsivo e centralizado.
R4	Tema escuro de estilo retro-arcade com elementos em tons neon.
R5	Placar com pontuação atual e melhor pontuação persistida no navegador.
R6	Tela de fim de jogo com botão de reinício.
R7	Controle por setas do teclado e por teclas W, A, S e D.
R8	Botões direcionais na tela para uso em dispositivos de toque.
R9	Bloqueio de inversão instantânea de direção.
R10	Alvo sorteado em posição aleatória, nunca sobre o corpo do jogador.
R11	Fim de jogo por colisão com a borda do tabuleiro ou com o próprio corpo.
R12	Crescimento do jogador e aumento de pontuação a cada alvo consumido.
R13	Código estruturado, comentado e limpo.
R14	Laço de jogo eficiente, com <code>requestAnimationFrame</code> ou temporizador controlado.

Fonte: adaptado de `docs/memorial.txt` (solicitação original do produto).

Quadro 4.2 – Etapas do procedimento e artefatos produzidos

Etapas	Nome	Artefatos produzidos
E1	Auditoria e compreensão	Classificação do material disponível; identificação de oito lacunas no material de origem do framework.
E2	Especificação	Especificação com 15 requisitos funcionais, 9 não funcionais, 12 critérios de aceitação, contratos de dados e casos de borda.
E3	Projeto e implementação	Documento com 7 decisões arquiteturais e o artefato de código único.
E4	Verificação	Protocolo de teste, execução das verificações e registro de evidências com resultado por critério.

Fonte: elaborado pelo autor (2026).

4.4 Instrumentos

Foram usados quatro instrumentos, todos disponíveis no ambiente sem instalação adicional.

Inspeção estática. Busca textual no artefato para confirmar propriedades estruturais que não dependem de execução, como a ausência de referências de rede e a existência de um único bloco de estilo e um único bloco de script.

Automação de navegador. Controle programático de um navegador real para enviar eventos de teclado e de apontador e para inspecionar o estado observável da página. Esse instrumento permite verificar comportamento sem instrumentar o código do produto — ponto relevante, porque instrumentar o produto para testá-lo alteraria o objeto de estudo.

Leitura de pixels da superfície de desenho. Amostragem dos pixels do elemento de desenho para localizar a célula do alvo e a posição da cabeça do jogador. Essa técnica permitiu implementar um agente de teste que joga o jogo de forma autônoma, tornando possível verificar crescimento, pontuação e velocidade sem intervenção manual e sem acesso à estrutura interna do programa.

Captura de tela. Registro visual dos estados de interface, usado como evidência complementar qualitativa.

4.5 Ambiente experimental

O Quadro 4.3 registra o ambiente, conforme exigido para reprodutibilidade. Os itens marcados como não registrados não foram anotados durante a execução e não são reconstruídos aqui por estimativa.

Quadro 4.3 – Ambiente experimental registrado

Item	Registro
Sistema operacional	Linux
Forma de execução	Arquivo local, protocolo de arquivo, sem servidor
Motor de navegador	Motor Chromium
Versão exata do navegador	[DADO NECESSÁRIO — não registrado durante a execução]
Resolução da janela	Área de conteúdo de aproximadamente 1200 px de largura; elemento de desenho com 517,5 px de lado
Razão de pixels do dispositivo	1 (não emulado)
Armazenamento local	Disponível e persistente entre recarregamentos
Idioma da interface	Português do Brasil
Preferência de movimento reduzido	[DADO NECESSÁRIO — estado não registrado]
Duração das sessões de medição	Janela de 150 s para a medição de velocidade

Fonte: elaborado pelo autor (2026).

4.6 Variáveis

Variável independente: a presença do processo orientado por especificação, com critérios de aceitação definidos antes da implementação.

Variáveis dependentes: (i) fração A de critérios de aceitação satisfeitos, definida na Equação (2.2); (ii) cobertura de rastreabilidade, definida como a fração de requisitos com critério de aceitação associado; (iii) intervalo médio entre passos de simulação, definido na Equação (4.1).

Variáveis de controle: as constantes do alvo (dimensões da grade, intervalo inicial, intervalo mínimo, passo de redução, pontos por item), fixadas no artefato e registradas na Tabela 3.

O intervalo entre passos de simulação é o principal alvo de medição quantitativa porque é a única grandeza de comportamento contínuo cujo valor alvo está declarado na especificação e que é observável externamente sem instrumentar o produto.

4.7 Métricas

A Tabela 2 define as métricas operacionais, a forma de coleta e a limitação de cada uma.

Tabela 2 – Métricas operacionais, coleta e limitação

Métrica	Coleta	Limitação
A (cobertura de verificação)	Contagem de critérios com resultado observado favorável sobre o total	Mede conformidade com a especificação, não adequação ao uso
Rastreabilidade de requisitos	Contagem de requisitos com critério de aceitação associado sobre o total	Não avalia a qualidade do critério, apenas sua existência
t (intervalo entre passos)	Marcações de tempo do observador entre mudanças sucessivas da posição da cabeça	A granularidade do observador adiciona erro positivo; o valor medido é um limite superior
Comprimento do jogador	Contagem de células ocupadas na superfície de desenho	Depende da detecção correta de cor
Colisões indevidas	Contagem de amostras em que o alvo coincidiu com célula do corpo	Limitada ao número de amostras coletadas

Fonte: elaborado pelo autor (2026).

O erro positivo do observador na medição de t merece destaque, porque justifica o tratamento adotado. Seja ε o atraso introduzido pelo ciclo de observação. O intervalo medido é $t_{\text{med}} = t_{\text{real}} + \varepsilon$. Como ε é aproximadamente constante ao longo de uma janela, a comparação entre janelas preserva a *diferença* mas não o valor absoluto. Por isso, os valores absolutos medidos não são tratados como estimativas do parâmetro do alvo; o que se afirma é a *ordem* e a *direção* da variação, conforme a Equação (4.1).

4.8 Tratamento dos dados

Para cada mudança de posição da cabeça do jogador registrou-se um par (instante, pontuação corrente). O intervalo médio entre passos em uma faixa de pontuação foi estimado por:

$$E[T] = \frac{1}{m-1} \sum_{i=2}^m (\tau_i - \tau_{i-1}) \quad (4.1)$$

em que τ_i é o instante da i -ésima mudança de posição e m é o número de mudanças observadas na faixa. A média, e não a mediana, foi adotada como tratamento principal porque o erro de observação ε é aditivo e aproximadamente constante: a média o absorve de forma previsível, enquanto a mediana o distribui de modo dependente da amostra.

A estatística descritiva completa por faixa — contagem, média, mínimo e máximo — está reportada no Capítulo 8. Não se aplicou teste de hipótese, pelo tamanho da amostra e pela natureza de estudo de caso; essa escolha é retomada em 4.10.

4.9 Ética e integridade

O trabalho não envolveu seres humanos como sujeitos de pesquisa, não coletou dados pessoais e não utiliza dados sensíveis; a apreciação por comitê de ética não se aplica. O produto armazena apenas um valor numérico de recorde no armazenamento local do navegador de quem o executa.

Declara-se explicitamente a política de integridade adotada: nenhum dado, medição, resultado ou referência foi estimado, interpolado ou produzido de memória. Informação ausente recebeu marcador explícito e foi registrada em `MONOGRAFIA_PENDENCIAS.md`. As referências bibliográficas foram declaradas sem identificadores não verificados — sem ISBN ou DOI inventados — e exigem conferência contra a fonte original antes da entrega final.

4.10 Ameaças à validade

A classificação segue Wohlin et al. (2012) e Runeson e Höst (2009). O Quadro 4.4 enumera as ameaças identificadas, a categoria e a mitigação efetivamente aplicada.

Duas ameaças não foram mitigadas: a ausência de grupo de controle (AV-1) e a ausência de repetição com tratamento estatístico (AV-6). Ambas exigiriam trabalho experimental adicional. Registrá-las explicitamente é preferível a apresentar os resultados como se essas limitações não existissem.

Quadro 4.4 – Ameaças à validade identificadas e mitigações

ID	Ameaça	Categoria e mitigação
AV-1	Ausência de grupo de controle que executou o mesmo alvo sem o processo proposto	<i>Validade interna.</i> Não mitigada. Sem controle não é possível atribuir o resultado ao processo em vez de à competência do executor ou à simplicidade do alvo. Mitigação parcial: registrar a lacuna (P-14).
AV-2	Estudo de caso único em um domínio com estado discreto e regras bem definidas	<i>Validade externa.</i> Generalização limitada a domínios de complexidade comparável. Mitigação: declarar explicitamente o escopo de generalização.
AV-3	Autoria do executante e do observador coincidirem	<i>Validade de construto.</i> Risco de confirmação. Mitigação: a evidência é produzida por execução automatizada e por inspeção textual reproduzível, não por julgamento do autor.
AV-4	A métrica de rastreabilidade avalia existência de critério, não sua qualidade	<i>Validade de construto.</i> Um critério mal formulado contaria como cobertura. Mitigação: não mitigada; registrada como limitação da métrica.
AV-5	Erro positivo na medição do intervalo por ciclo de observação	<i>Validade de conclusão.</i> Afeta valores absolutos. Mitigação: reportar apenas a direção da variação e declarar o viés.
AV-6	Amostra temporal única por faixa, sem repetição nem intervalo de confiança	<i>Validade de conclusão.</i> Não permite inferência estatística. Mitigação: não mitigada; repetição registrada como dado necessário (P-11).
AV-7	Deteção de posições por cor, sujeita a erro em pixels de borda	<i>Validade de construto.</i> Mitigação: critérios de cor restritivos e verificação de que a célula do alvo nunca foi classificada como corpo em 25 amostras.

Fonte: elaborado pelo autor (2026), a partir de Wohlin et al. (2012) e Runeson e Höst (2009).

5 O FRAMEWORK SDD AGÊNTICO

Este capítulo descreve o processo proposto: sua arquitetura, os agentes, as instruções permanentes, as habilidades de domínio, o fluxo adaptativo, os portões de verificação, a definição de concluído, o protocolo de contexto e as condições de parada. O framework é a variável independente do estudo.

5.1 Princípios de projeto

O framework foi projetado a partir de oito princípios operacionais, adotados como regras de conduta do executor:

1. Planejar antes de editar.
2. Verificar antes de assumir.
3. Implementar pouco por vez.
4. Testar imediatamente.
5. Registrar decisões importantes.
6. Nunca esconder incerteza.
7. Nunca fabricar evidência.
8. Nunca declarar validação física sem validação física.

Os princípios 6 a 8 são os mais consequentes para o resultado deste trabalho. Eles convertem uma postura epistemológica em regra operacional: a ausência de dado não é uma falha a ser disfarçada, mas um estado a ser declarado. O item 8 é específico ao domínio de sistemas embarcados do material de origem e estabelece que nenhum nível de evidência pode ser promovido por analogia — teste em simulador não promove um resultado a validade em bancada.

5.2 Motivação: lacunas do material de origem

O ponto de partida foi uma avaliação crítica de um conjunto prévio de artefatos de processo. Essa avaliação identificou oito oportunidades de melhoria, listadas no Quadro 5.1 com o tratamento dado a cada uma no framework proposto.

Quadro 5.1 – Lacunas identificadas no material de origem e tratamento no framework

#	Lacuna identificada	Tratamento no framework proposto
L1	Fluxo linear demais, com cadeia completa mesmo para alterações pequenas	Roteamento por complexidade e risco, com quatro trilhas (Seção 5.7)
L2	Ausência de um portão independente de verificação	Agente dedicado a responder se há evidência suficiente para declarar conclusão
L3	Economia de contexto não era regra operacional permanente	Conjunto de instruções permanentes com dez regras de contexto
L4	Falta de separação explícita entre estado real e memória da conversa	Protocolo de contexto com ordem de precedência de fontes
L5	Rastreabilidade definida em conceito, não operacionalizada	Matriz de rastreabilidade com formato fixo e regra de evidência obrigatória
L6	Risco de duplicação entre documentos de estado e de transferência	Definição de papel único por documento; transferência só quando há continuidade
L7	Ausência de mecanismo explícito de parada	Seis condições de parada obrigatórias
L8	Definição de concluído dispersa em vários documentos	Lista única de verificação de encerramento com quatorze itens

Fonte: elaborado pelo autor (2026) a partir da avaliação crítica do material de origem.

5.3 Arquitetura em camadas

O framework organiza-se em cinco camadas com responsabilidades distintas, como ilustra a Figura 2. A separação é funcional: cada camada responde a uma pergunta diferente, e a violação da fronteira entre camadas é a origem mais comum de degradação de processo.

Figura 2 – Camadas do framework SDD agêntico e responsabilidade de cada uma



Fonte: elaborado pelo autor (2026) a partir da estrutura do repositório.

Camada de princípios. Contém o que não muda com a tarefa: a constituição e as regras permanentes. É a camada mais estável e a de maior precedência.

Camada de instruções. Contém regras operacionais de aplicação ampla: economia de contexto, controle de mudanças, artefatos e engenharia de domínio. Instruções são carregadas por padrão de arquivo afetado, e não por decisão discricionária.

Camada de habilidades. Contém conhecimento de domínio acionado sob demanda: produção de especificação, desenvolvimento embarcado, artigo científico, monografia e documentação de projeto. Habilidades não são carregadas preventivamente.

Camada de agentes. Contém quinze papéis especializados, cada um com entrada esperada, artefatos que pode ler, artefatos que pode alterar e portão de saída.

Camada de artefatos de controle. Contém o estado do trabalho: especificação, decisões, tarefas, estado, transferência e matriz de rastreabilidade. É a única camada que persiste entre sessões.

5.4 Agentes

Os quinze agentes distribuem-se em quatro grupos funcionais, conforme o Quadro 5.2.

A separação entre o agente que implementa (05) e os que verificam (06, 07, 13) é deliberada e corresponde ao princípio clássico de independência entre construção e verificação. O agente `code-reviewer` e o `test-engineer` avaliam; o `verification-gate` decide. Concentrar as três funções em um único papel reproduziria o problema que a separação pretende resolver.

5.5 Instruções permanentes

Quatro conjuntos de instruções são aplicados por padrão de arquivo afetado. Essa forma de aplicação — automática e baseada em caminho — remove a decisão de carregar ou não carregar da esfera discricionária do executor, o que é relevante quando o executor é um agente.

A instrução de controle de mudanças contém uma proibição que merece destaque: é vedado alterar testes com o objetivo de fazê-los passar. Essa é a regra que impede a degradação mais comum de processos de verificação — a adaptação retroativa do critério ao resultado obtido.

Quadro 5.2 – Agentes do framework, por grupo funcional

#	Agente	Responsabilidade
<i>Grupo A — Roteamento e planejamento</i>		
00	<code>task-router</code>	Classifica a solicitação, estima complexidade e risco e encaminha ao menor fluxo suficiente
01	<code>product-planner</code>	Define escopo, objetivos, restrições, riscos e critérios de sucesso
02	<code>requirements-engineer</code>	Redige requisitos funcionais e não funcionais, regras de negócio, casos de uso e critérios de aceitação
03	<code>architect</code>	Define componentes, interfaces, dados, tecnologias e decisões arquiteturais
04	<code>task-planner</code>	Decompõe requisitos e arquitetura em tarefas pequenas, ordenadas e verificáveis
<i>Grupo B — Execução</i>		
05	<code>implementer</code>	Executa a tarefa aprovada seguindo requisitos, arquitetura e padrões do projeto
<i>Grupo C — Verificação</i>		
06	<code>code-reviewer</code>	Avalia corretude, qualidade, manutenibilidade, testes e aderência à arquitetura
07	<code>test-engineer</code>	Define a estratégia de teste e cria testes de unidade, integração, sistema e ponta a ponta
08	<code>security-reviewer</code>	Avalia risco em código, arquitetura, dependências, configuração e tratamento de dados
13	<code>verification-gate</code>	Responde se há evidência suficiente para declarar a tarefa concluída
<i>Grupo D — Encerramento e continuidade</i>		
09	<code>documentation</code>	Mantém a documentação técnica e de uso sincronizada
10	<code>integration-agent</code>	Valida a integração entre módulos, serviços, hardware e interfaces
11	<code>release-agent</code>	Verifica qualidade, documentação, versionamento e critérios de aceite antes da liberação
12	<code>project-orchestrator</code>	Coordena os demais agentes e preserva a rastreabilidade do roteamento à transferência
14	<code>handoff-manager</code>	Produz transferência compacta e verificável entre agentes ou sessões

Fonte: elaborado pelo autor (2026) a partir de `.github/agents/`.

5.6 Habilidades de domínio

Cinco habilidades concentram conhecimento especializado, carregado apenas quando a tarefa correspondente está ativa:

- **Criação de especificação** — estrutura de especificação otimizada para consumo por agente, com distinção entre requisito, restrição e recomendação.

Quadro 5.3 – Instruções permanentes do framework

Instrução	Aplicação	Conteúdo
Economia de contexto	Todos os arquivos	Dez regras de inspeção incremental; proíbe varredura proativa; obriga leitura de estado antes de alterar
Engenharia embarcada	Código C, C++, Rust, Python e assembly	Restrições de memória, tempo real, concorrência e segurança específicas do domínio
Artefatos SDD	Especificações, documentação e arquivos de estado	Convenções de identificador (FR, NFR, CA, T, ADR) e regra de evidência obrigatória
Controle de mudanças	Todos os arquivos	Escopo, menor alteração suficiente, proibição de alterar testes para fazê-los passar e verificação posterior obrigatória

Fonte: elaborado pelo autor (2026) a partir de [.github/instructions/](https://github.com/author/instructions/).

- **Desenvolvimento embarcado** — especificação e implementação de firmware, com níveis de evidência e fluxo adaptativo.
- **Artigo científico em LaTeX** — produção com rigor metodológico e referências verificáveis.
- **Monografia de engenharia em LaTeX** — produção com conformidade ABNT, diagramas e projeto compilável.
- **Documentação de projeto** — criação e manutenção do arquivo de apresentação do repositório.

O carregamento sob demanda é o mecanismo que resolve a tensão identificada na Seção 2.10: conhecimento especializado existe sem ocupar contexto quando não é necessário.

5.7 Fluxo adaptativo

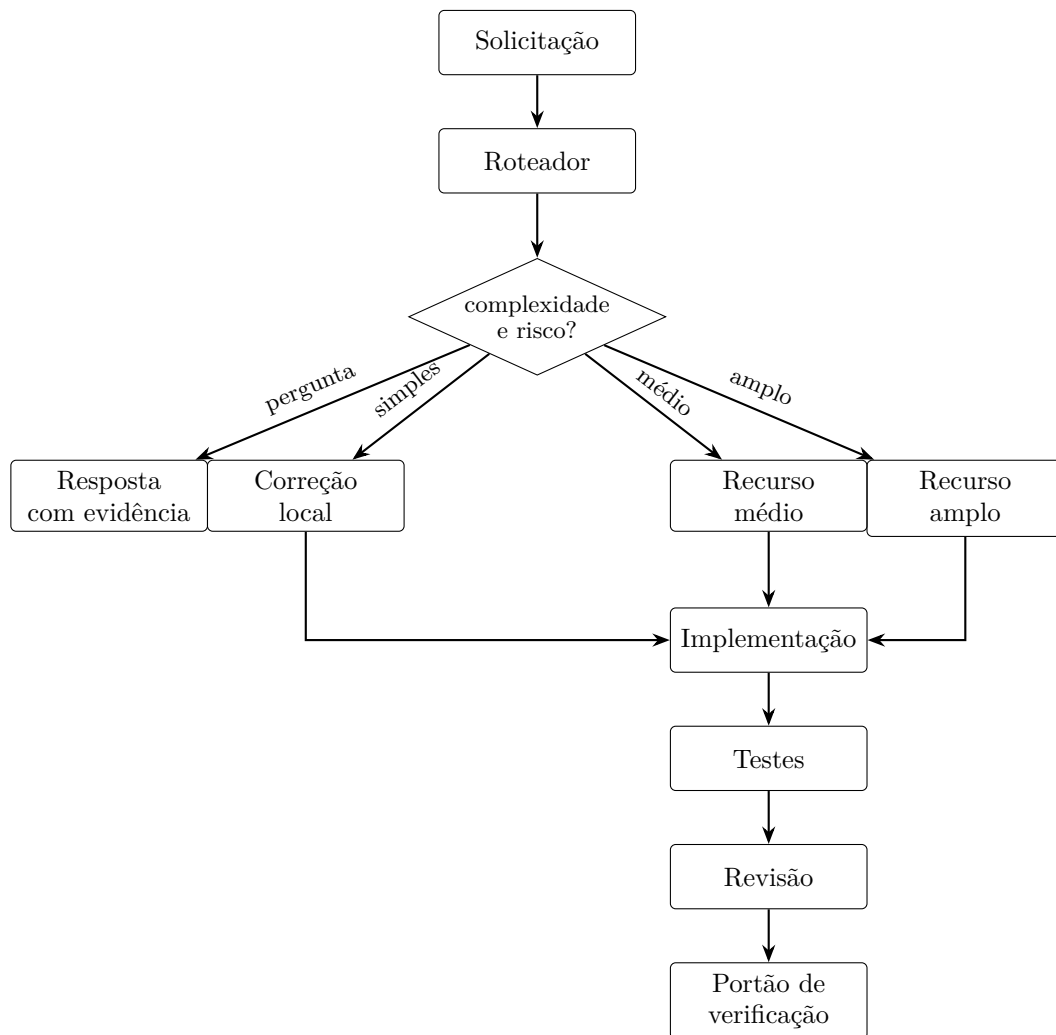
O fluxo não aplica a cadeia completa a toda solicitação. O roteador classifica a complexidade e o risco e seleciona a trilha mínima suficiente, como ilustra a Figura 3. Essa decisão responde diretamente à lacuna L1.

O Quadro 5.4 descreve as quatro trilhas e o critério de elevação entre elas.

5.8 Portões de verificação

Seis portões pontuam o fluxo, conforme a Figura 4 e o Quadro 5.5.

Figura 3 – Fluxo agêntico adaptativo: classificação por complexidade e risco



Fonte: elaborado pelo autor (2026) a partir de docs/agenttic/WORKFLOW.md. A trilha de liberação, aplicada após o portão de verificação, foi omitida para preservar a legibilidade.

O portão G1 é o que garante a precedência do critério sobre a implementação: sem requisito e critério identificados, o fluxo não avança para a execução. O portão G4 é o que impede a conclusão sem evidência.

5.9 Definição de concluído

A definição de concluído consolida, em uma única lista de quatorze itens, o que estava disperso em vários documentos de origem (lacuna L8):

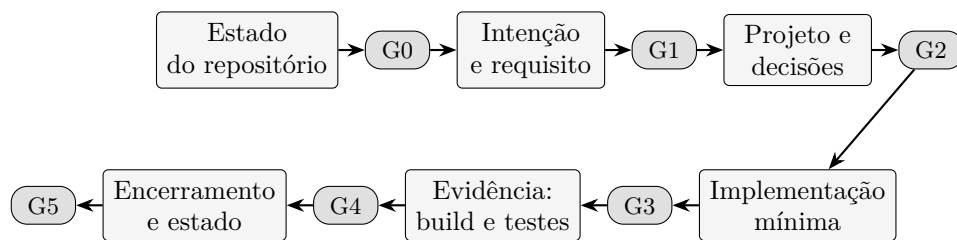
1. requisito relacionado identificado; 2. critérios de aceitação verificados; 3. estado do repositório inspecionado antes da alteração; 4. alteração no menor escopo necessário; 5. compilação do alvo executada; 6. testes executados; 7. análise estática executada quando aplicável; 8. consumo de memória e tempo verificados quando aplicável; 9. documentação

Quadro 5.4 – Trilhas do fluxo adaptativo e critério de elevação

Trilha	Sequência de agentes	Critério de elevação
Pergunta	Resposta fundamentada, sem alteração de artefato	Necessidade de alterar qualquer arquivo
Correção local	Roteador, executor, portão de verificação	Alteração de comportamento observável
Recurso médio	Roteador, requisitos, executor, teste, revisão, verificação	Necessidade de decisão arquitetural
Recurso amplo	Planejamento, requisitos, arquitetura, tarefas, execução, teste, revisão, segurança, verificação, documentação, integração	—
Liberação	Verificação, segurança, documentação, integração, liberação	—

Fonte: elaborado pelo autor (2026).

Figura 4 – Pipeline SDD com os seis portões de verificação



Fonte: elaborado pelo autor (2026) a partir de docs/agentico/WORKFLOW.md. O portão G2 é aplicado apenas quando a complexidade exige decisão arquitetural.

Quadro 5.5 – Portões de verificação do framework

Portão	Nome	Condição de passagem
G0	Estado	Estado real do repositório inspecionado antes de qualquer alteração
G1	Intenção	Requisito e critério de aceitação identificados
G2	Projeto	Decisão arquitetural registrada quando a complexidade exige
G3	Implementação	Alteração mínima, rastreável e restrita ao escopo
G4	Evidência	Compilação, testes e validações aplicáveis executados
G5	Encerramento	Documentação, rastreabilidade, estado e riscos atualizados

Fonte: elaborado pelo autor (2026) a partir de docs/agentico/WORKFLOW.md.

atualizada; 10. rastreabilidade atualizada; 11. arquivo de estado atualizado se houve mudança significativa; 12. arquivo de transferência atualizado apenas se houver continuidade; 13. riscos residuais registrados; 14. nenhuma alteração não relacionada incorporada.

Os itens 5, 6, 7 e 8 são “quando aplicável”. Essa qualificação é deliberada: em um artefato sem compilação — como o produto deste estudo de caso, que é interpretado

diretamente pelo navegador — não existe etapa de compilação, e exigi-la produziria um item permanentemente não cumprido. A condição de aplicabilidade precisa ser declarada, e não presumida.

5.10 Protocolo de contexto

O protocolo de contexto estabelece a ordem de precedência entre fontes de informação:

1. estado real do repositório e do sistema de controle de versão;
2. princípios permanentes;
3. especificação do recurso;
4. registro de decisões arquiteturais;
5. tarefas;
6. código atual;
7. testes e evidências;
8. arquivos de estado e de transferência;
9. histórico da conversa.

A regra central é: *o histórico da conversa pode fornecer intenção, mas nunca deve substituir a leitura do estado atual do repositório*. Essa regra endereça a lacuna L4 e é a defesa direta contra a falha mais insidiosa em fluxos executados por agentes — tratar uma versão lembrada de um arquivo como se fosse a versão presente. A posição do histórico no último nível da hierarquia é a expressão formal dessa defesa.

5.11 Condições de parada

Seis condições obrigam a interrupção do fluxo e o pedido de decisão, em vez de prosseguimento sob suposição:

1. informação essencial de hardware ausente;
2. duas especificações em conflito;
3. necessidade de mudança arquitetural não aprovada;

4. risco de perda de dados ou de persistência;
5. impacto de segurança não especificado;
6. teste obrigatório que não pode ser executado quando o resultado é necessário para liberar a tarefa.

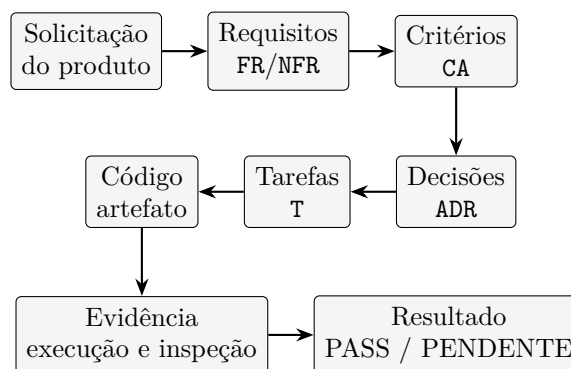
A segunda condição é a que produz mais consequências neste trabalho, porque uma situação dessa natureza foi de fato encontrada: o repositório descreve um sistema embarcado e contém, simultaneamente, um produto de página web. Diante do conflito, a conduta adotada foi registrar a divergência e declarar a escolha, em vez de produzir documentação sobre um sistema que não existe no repositório (Seção 1.6).

5.12 Matriz de rastreabilidade

A matriz operacionaliza a rastreabilidade (lacuna L5) com sete colunas fixas: requisito, critério, tarefa, código, teste, evidência e estado. Três regras a governam: não marcar como aprovado sem evidência; acompanhar toda pendência de justificativa e risco residual; e propagar toda alteração de requisito para tarefa, teste e documentação.

Figura 5 representa a cadeia de rastreabilidade aplicada ao estudo de caso.

Figura 5 – Cadeia de rastreabilidade do requisito à evidência



Fonte: elaborado pelo autor (2026). A ausência de critério rompe a cadeia entre requisito e evidência: é exatamente o caso dos dez requisitos marcados como **SEM CA** no Apêndice A.

5.13 Síntese do capítulo

O framework proposto é um protocolo de controle: cinco camadas, quinze agentes, quatro conjuntos de instruções, cinco habilidades, seis portões, catorze itens de conclusão e seis condições de parada. Seus dois mecanismos distintivos são a *precedência do critério* sobre a implementação e a *proibição de evidência fabricada*. O Capítulo 6 mostra o primeiro em operação; o Capítulo 8, o segundo.

6 ESPECIFICAÇÃO DO SISTEMA-ALVO

Este capítulo apresenta a especificação do sistema-alvo produzida antes de qualquer linha de implementação. A ordem é deliberada: primeiro os requisitos funcionais, depois os não funcionais, depois as restrições, depois as decisões arquiteturais e, por fim, os critérios de aceitação. Os critérios de aceitação fecham o capítulo porque são derivados dos requisitos, e não o contrário.

6.1 Contexto de especificação

A entrada do processo foi um documento de solicitação do produto, com catorze requisitos expressos em linguagem natural (Quadro 4.1). A tarefa da etapa de especificação foi transformar essa entrada em requisitos identificados, verificáveis e não ambíguos, sem ampliar o escopo.

Três decisões de tradução foram tomadas nessa etapa e merecem registro, porque alteraram o conteúdo do que foi especificado em relação ao que foi pedido.

Ausência de recursos bônus. A solicitação original mencionava uma seção opcional de recursos adicionais — efeitos sonoros, níveis de dificuldade, itens especiais — e declarava que tais recursos só entrariam se fossem explicitados. Como nenhum foi explicitado, nenhum foi especificado. A decisão evita a expansão de escopo por iniciativa do executor, que é um modo comum de desvio de requisito.

Requisitos não funcionais não pedidos. Foram especificados requisitos não funcionais que a solicitação não mencionava: nitidez em telas de alta densidade (NFR-006), prevenção de rolagem de página por teclas de direção (NFR-007) e suporte a preferência de movimento reduzido (NFR-009). Esses itens derivam de boas práticas de plataforma web e de acessibilidade (World Wide Web Consortium (2023)), e sua inclusão é uma decisão do executor, não uma exigência do solicitante. Ela é declarada aqui por transparência.

Requisitos de plataforma promovidos a requisito. A solicitação pedia suporte a toque; a especificação converteu isso em um requisito funcional com componente de interface identificável (FR-004) e em um critério de aceitação próprio (CA-008), tornando o item verificável.

6.2 Requisitos funcionais

Os quinze requisitos funcionais estão no Quadro 6.1. Cada um descreve comportamento observável do sistema.

Quadro 6.1 – Requisitos funcionais do sistema-alvo

ID	Requisito
FR-001	Entregar o produto como um único arquivo de página web, com estilo e script embutidos, sem arquivos auxiliares.
FR-002	Renderizar tabuleiro, jogador e alvo em superfície de desenho vetorial bidimensional.
FR-003	Permitir controle por teclas de direção e pelas teclas W, A, S e D, em ambas as caixas.
FR-004	Exibir quatro botões direcionais na tela, operáveis por toque e por apontador.
FR-005	Impedir inversão instantânea de direção.
FR-006	Sortear o alvo em célula livre da grade, nunca sobre o corpo do jogador.
FR-007	Crescer uma célula e somar dez pontos a cada alvo consumido.
FR-008	Encerrar a partida ao colidir com a borda do tabuleiro ou com o próprio corpo.
FR-009	Exibir pontuação atual e melhor pontuação.
FR-010	Ler e gravar a melhor pontuação no armazenamento local, tolerando indisponibilidade sem interromper o jogo.
FR-011	Exibir tela de fim de jogo com pontuação final e botão de reinício que restaura o estado inicial.
FR-012	Iniciar a movimentação somente após a primeira entrada direcional válida.
FR-013	Permitir pausar e retomar, inclusive por perda de foco da janela.
FR-014	Usar laço de animação com acumulador de tempo, desacoplando simulação de taxa de quadros.
FR-015	Aumentar progressivamente a velocidade conforme a pontuação, respeitando intervalo mínimo.

Fonte: elaborado pelo autor (2026) a partir de `.specs/features/snake-game/spec.md`.

6.3 Requisitos não funcionais

Os nove requisitos não funcionais estão no Quadro 6.2. O mapeamento nas características de qualidade da ISO/IEC 25010 foi apresentado na Tabela 1.

6.4 Restrições e diretrizes

Quatro restrições limitam o espaço de solução. CON-001 exige artefato único, proibindo arquivos de estilo ou script separados. CON-002 admite apenas CSS e JavaScript nativos, sem transpilação. CON-003 exige centralizar grade e constantes de jogo em um único objeto de configuração. CON-004 proíbe usar temporizador fixo para a simulação.

A restrição CON-004 é a mais consequente, porque interdita uma solução que funcionaria: um temporizador de intervalo fixo dispararia a simulação com regularidade suficiente para o jogo. A justificativa é a acumulação de atrasos: temporizadores fixos não se adaptam à carga do navegador e tendem a acumular deriva, além de não sincronizar

Quadro 6.2 – Requisitos não funcionais do sistema-alvo

ID	Requisito
NFR-001	Zero dependências externas: sem bibliotecas, sem quadros de trabalho, sem rede, sem fontes ou imagens remotas.
NFR-002	Layout moderno, limpo, centralizado horizontal e verticalmente.
NFR-003	Tema escuro de estilo retro-arcade, com elementos em tons neon.
NFR-004	Adaptar-se a larguras de tela de 320 px a 1920 px, sem rolagem horizontal indesejada.
NFR-005	Código comentado, organizado em seções, com identificadores em inglês e comentários em português.
NFR-006	Ajustar as dimensões de desenho à razão de pixels do dispositivo para nitidez em telas de alta densidade.
NFR-007	Tratar entrada sem bloquear o laço de animação; impedir a rolagem da página provocada pelas teclas de direção.
NFR-008	Carregar e iniciar sem erros no console do navegador.
NFR-009	Respeitar a preferência de movimento reduzido do sistema operacional.

Fonte: elaborado pelo autor (2026) a partir de `.specs/features/snake-game/spec.md`.

com a taxa de atualização do dispositivo. A alternativa adotada — laço de animação com acumulador — é detalhada no Capítulo 7.

6.5 Decisões arquiteturais

Sete decisões foram registradas no formato de problema, alternativa escolhida, alternativas descartadas e justificativa. O Quadro 6.3 as resume.

6.6 Contratos de dados

A especificação declara quatro contratos, o que viabiliza a verificação por inspeção e a implementação sem ambiguidade de nomes: o objeto de configuração com sete campos; o objeto de estado com dez campos; a chave de armazenamento local com valor inteiro; e a tabela de eventos de entrada com a origem, o evento e a ação correspondente.

A Tabela 3 registra os valores concretos das constantes, conforme fixados na especificação e implementados no artefato.

6.7 Regra de evolução da velocidade

A relação entre pontuação e intervalo entre passos é a única regra do alvo com comportamento contínuo observável, e por isso é central para a verificação. Sua forma é:

Quadro 6.3 – Decisões arquiteturais do sistema-alvo

ID	Decisão	Justificativa e alternativa descartada
ADR-001	Desenho vetorial em tela única	Evita centenas de elementos estruturais e mantém estabilidade de quadro. Descartado: um elemento por célula.
ADR-002	Simulação por passo discreto com acumulador	Desacopla lógica de taxa de quadros. Descartado: temporizador de intervalo fixo (vedado por CON-004).
ADR-003	Fila curta de entrada com validação	Evita inversão e perda de comando em passos rápidos. Descartado: aplicação imediata da última tecla.
ADR-004	Remover a cauda antes de testar colisão	Permite ocupar a célula que a cauda acabou de liberar. Descartado: testar colisão com o corpo completo.
ADR-005	Objeto único de configuração	Centraliza grade, tempos, pontos e chave de armazenamento (CON-003).
ADR-006	Persistência tolerante a falha	Acesso ao armazenamento envolto em tratamento de exceção (FR-010).
ADR-007	Tema por variáveis de estilo	Consistência visual e suporte a movimento reduzido.

Fonte: elaborado pelo autor (2026) a partir de `.specs/features/snake-game/design.md`.

$$t(e) = \max(t_{\min}, t_0 - e \cdot \Delta), \quad e = \left\lfloor \frac{s}{p} \right\rfloor \quad (6.1)$$

Tabela 3 – Constantes do alvo, conforme especificação e implementação

Constante	Valor	Unidade	Papel
Colunas da grade	24	células	Dimensão horizontal
Linhas da grade	24	células	Dimensão vertical
Comprimento inicial	4	células	Tamanho inicial do jogador
Intervalo inicial t_0	140	ms	Passo de simulação no início
Intervalo mínimo $t_{\min}$	68	ms	Piso do passo de simulação
Passo de redução Δ	4	ms	Redução por item consumido
Pontos por item	10	pontos	Incremento de pontuação
Tamanho máximo da fila	2	entradas	Capacidade da fila de direção
Teto de passos por quadro	5	passos	Proteção contra acumulação
Chave de armazenamento	—	texto	Identificador do registro local

Fonte: elaborado pelo autor (2026) a partir de `spec.md` e `index.html`.

em que e é o número de itens consumidos, s é a pontuação corrente, $p = 10$ é a pontuação por item, $t_0 = 140$ ms, $t_{\min} = 68$ ms e $\Delta = 4$ ms. Substituindo p e Δ , obtém-se a forma equivalente

$$t(s) = \max\left(68, 140 - \frac{2s}{5}\right) \text{ ms.} \quad (6.2)$$

A saturação ocorre em $e = 18$ itens, isto é, $s = 180$ pontos, quando $140 - 18 \cdot 4 = 68$ ms. A partir desse ponto, a velocidade permanece constante por construção. Essa predição é verificável e foi verificada indiretamente pela medição reportada na Seção 8.5.

6.8 Critérios de aceitação

Os doze critérios de aceitação estão no Quadro 6.4. Cada critério é enunciado de forma que um observador externo possa decidir se foi satisfeito, sem consultar o autor e sem acesso ao histórico de desenvolvimento.

Quadro 6.4 – Critérios de aceitação do sistema-alvo

ID	Critério
CA-001	Existe exatamente um arquivo, com estilo e script embutidos e nenhuma referência de rede a script, estilo ou fonte.
CA-002	Ao pressionar direita e depois esquerda no mesmo passo, o jogador continua à direita.
CA-003	Ao consumir o alvo, o comprimento aumenta em uma célula e a pontuação em dez pontos.
CA-004	A célula do alvo pertence à grade e não pertence a nenhuma célula do corpo.
CA-005	Ao atingir célula fora da grade, o estado passa a encerrado e a tela de fim de jogo é exibida.
CA-006	Ao atingir célula ocupada pelo corpo, o estado passa a encerrado.
CA-007	Ao superar o recorde anterior, o novo valor é gravado no armazenamento local e exibido na reinicialização.
CA-008	Ao acionar um botão direcional, o jogador muda de direção respeitando CA-002.
CA-009	No estado inicial, ao pressionar uma direção válida, o jogador começa a se mover.
CA-010	Em execução, ao pressionar a tecla de pausa, o estado passa a pausado e o movimento cessa; novo acionamento retoma.
CA-011	Após cinco segundos de execução, o console não apresenta erros nem exceções.
CA-012	Ao aumentar a pontuação, o intervalo entre passos diminui até o mínimo definido.

Fonte: elaborado pelo autor (2026) a partir de `.specs/features/snake-game/spec.md`.

6.9 Casos de borda declarados

Cinco casos de borda foram declarados na especificação antes da implementação. Declará-los antecipadamente é o que permite transformá-los em teste, em vez de descobri-los como defeito.

1. **Inversão no mesmo passo** — a segunda entrada é descartada e a direção permanece.
2. **Grade completamente ocupada** — o sorteio do alvo não encontra célula livre; o jogo encerra sem travar.
3. **Armazenamento indisponível** — a leitura lança exceção; o recorde permanece zero e o jogo continua.
4. **Duas teclas no mesmo passo** — a segunda direção é enfileirada e aplicada no passo seguinte, se não for oposta.
5. **Colisão com a cauda em movimento** — mover para a célula que a cauda acabou de liberar não encerra a partida, por consequência de ADR-004.

O item 5 é o mais sutil e é o único caso de borda que depende diretamente de uma decisão arquitetural, e não apenas da regra de jogo. A ordem entre remover a cauda e testar colisão determina se o comportamento é morte ou sobrevivência; sem a decisão registrada em ADR-004, dois implementadores produziram resultados diferentes a partir da mesma especificação de requisitos. Esse é um argumento concreto a favor do registro de decisões arquiteturais.

6.10 Síntese do capítulo

A especificação produziu 15 requisitos funcionais, 9 requisitos não funcionais, 4 restrições, 7 decisões arquiteturais, 4 contratos de dados, 12 critérios de aceitação e 5 casos de borda declarados. A implementação descrita no Capítulo 7 foi conduzida sob esse contrato.

7 IMPLEMENTAÇÃO

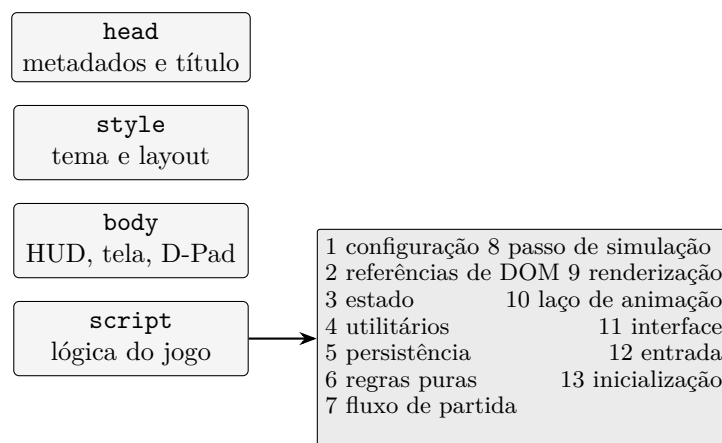
Este capítulo descreve a implementação do sistema-alvo: a estrutura do artefato, a máquina de estados da partida, o laço de animação, o algoritmo de um passo de simulação, a fila de entrada, o sorteio do alvo, a persistência, a renderização e as decisões de acessibilidade. Cada seção aponta para a decisão arquitetural e o requisito correspondentes.

7.1 Estrutura do artefato

O produto é um único arquivo de 1 063 linhas e 32 305 bytes, organizado em treze seções comentadas. A Figura 6 representa sua estrutura.

Figura 6 – Estrutura do artefato único e suas treze seções de código

Arquivo único: 1 063 linhas, 32 305 bytes



Fonte: elaborado pelo autor (2026) a partir de `index.html`. A restrição de artefato único (CON-001) impede a separação física entre módulos, de modo que o isolamento é obtido por funções de responsabilidade única dentro do mesmo arquivo.

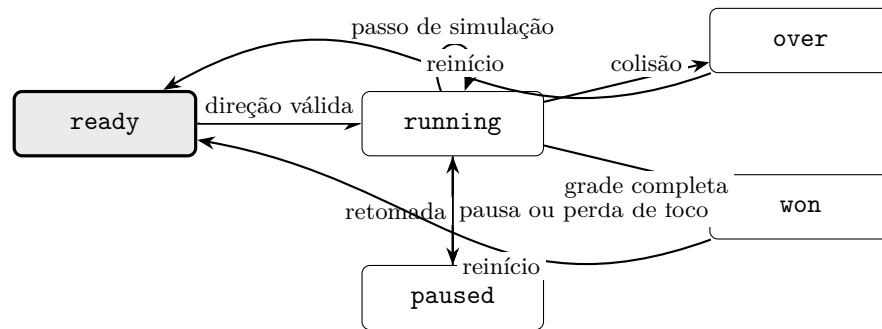
A organização em seções numeradas é a resposta a NFR-005 (código comentado e organizado) e cumpre função operacional, não apenas estética: como o processo de verificação precisa localizar trechos específicos por inspeção textual, a previsibilidade da estrutura reduz o esforço de verificação.

A restrição CON-001 (artefato único) tem uma consequência de projeto que vale explicitar: sem módulos separados, não há como isolar a lógica de simulação da lógica de apresentação por meio de fronteiras de arquivo. O isolamento foi obtido por meio de funções com responsabilidade única dentro do mesmo arquivo, e a separação é garantida por convenção, não por mecanismo. É um enfraquecimento real da arquitetura em relação a uma solução modular — aceito porque exigido pela especificação.

7.2 Máquina de estados da partida

A partida é modelada como máquina de estados finitos com cinco estados: **ready**, **running**, **paused**, **over** e **won**. A Figura 7 apresenta as transições.

Figura 7 – Máquina de estados da partida e transições



Fonte: elaborado pelo autor (2026) a partir de `index.html`, seção 7. O estado **won** não constava da solicitação original: ele foi criado para tratar o caso de borda “grade completamente ocupada”.

Duas decisões de modelagem merecem registro. A primeira é a existência do estado **won**, que não constava da solicitação original. Ele é necessário porque a especificação declarou o caso de borda “grade completamente ocupada” (Seção 6.9): sem esse estado, o sorteio do alvo retornaria um valor inválido e o jogo entraria em estado inconsistente. A modelagem do caso de borda forçou a criação de um estado. Esse é um exemplo concreto de como declarar casos de borda altera o projeto, e não apenas os testes.

A segunda é que o estado **ready** não é apenas decorativo. Ele existe porque a especificação exige que a movimentação comece somente após a primeira entrada direcional válida (FR-012). A consequência é que a primeira entrada precisa ser tratada de forma diferente das demais: como o jogador ainda não se moveu, qualquer direção cardinal é aceita, inclusive a oposta à direção inicial, e o corpo é reposicionado atrás da nova cabeça para evitar autocolisão imediata. Sem esse tratamento, pressionar “esquerda” na tela inicial mataria o jogador no primeiro passo.

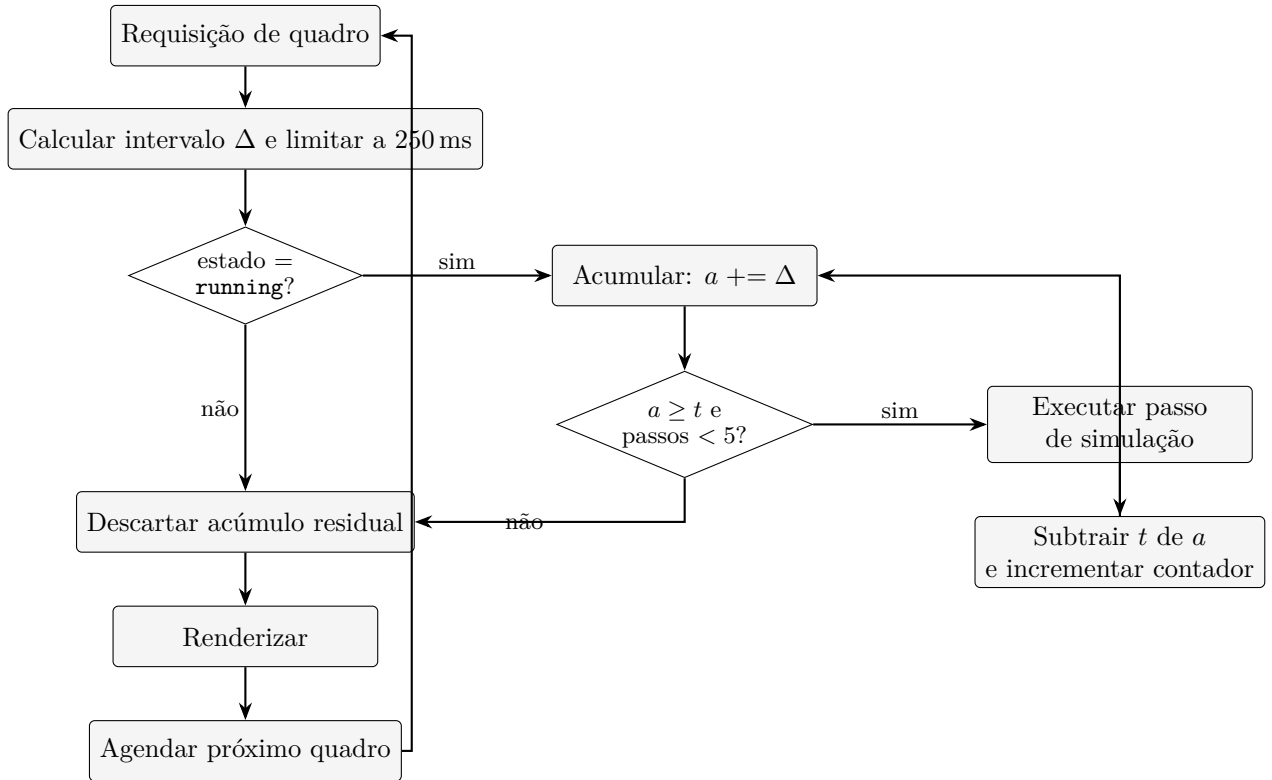
7.3 Laço de animação

O laço usa o mecanismo de requisição de quadro de animação (Mozilla Foundation (2024c)), com acumulador de tempo para separar a simulação da apresentação, conforme FR-014, CON-004 e ADR-002. A Figura 8 representa o fluxo.

O trecho central está no Quadro 7.1.

Três detalhes do trecho são decisões de engenharia, não acidentes de codificação.

Figura 8 – Laço de animação com acumulador de tempo e execução de passos



Fonte: elaborado pelo autor (2026) a partir de `index.html`, seção 10. O teto de cinco passos por quadro e o descarte do acúmulo residual são as duas defesas contra a espiral da morte.

Limitação do passo de tempo. A primeira linha limita o intervalo a 250 ms. Sem esse limite, um retorno de segundo plano produziria um intervalo de vários segundos e o jogador avançaria dezenas de células em um único quadro, sem possibilidade de reação. O limite converte uma anomalia de temporização em uma perda controlada de fluidez.

Teto de passos por quadro. A condição `steps < maxTicksPerFrame` limita a quantidade de simulações executadas em um mesmo quadro. Sem esse teto, a recuperação do atraso acumulado consumiria mais tempo de processamento do que o atraso original, ampliando a defasagem — fenômeno conhecido como espiral da morte. O teto rompe o ciclo ao custo de a simulação ficar momentaneamente atrás do tempo real.

Descarte do acúmulo residual. A linha final descarta o acúmulo quando ele excede o teto multiplicado pelo intervalo. É a segunda linha de defesa contra a mesma anomalia.

A Tabela 4 registra as grandezas temporais do laço.

Quadro 7.1 – Trecho do laço de animação: acúmulo de tempo e execução de passos (excertado)

```

1  const delta = Math.min(timestamp - state.lastTs, 250);
2  state.lastTs = timestamp;
3  state.elapsed += delta;
4
5  if (state.status === 'running') {
6    state.acc += delta;
7    let steps = 0;
8    while (state.acc >= state.stepMs && steps < CONFIG.maxTicksPerFrame) {
9      state.acc -= state.stepMs;
10     steps += 1;
11     tick();
12     if (state.status !== 'running') { state.acc = 0; break; }
13   }
14   if (state.acc > state.stepMs * CONFIG.maxTicksPerFrame) state.acc = 0;
15 } else {
16   state.acc = 0;
17 }

```

Fonte: `index.html`, seção 10.

Tabela 4 – Grandezas temporais do laço de animação

Grandeza	Valor	Observação
Intervalo inicial entre passos	140 ms	Aproximadamente sete passos por segundo
Intervalo mínimo	68 ms	Aproximadamente quinze passos por segundo
Teto de passo de tempo	250 ms	Limite do intervalo aceito em um quadro
Teto de passos por quadro	5	Proteção contra espiral da morte
Taxa de quadro alvo	—	Delegada ao navegador pelo mecanismo de requisição de quadro

Fonte: elaborado pelo autor (2026) a partir de `index.html`.

7.4 Algoritmo de um passo de simulação

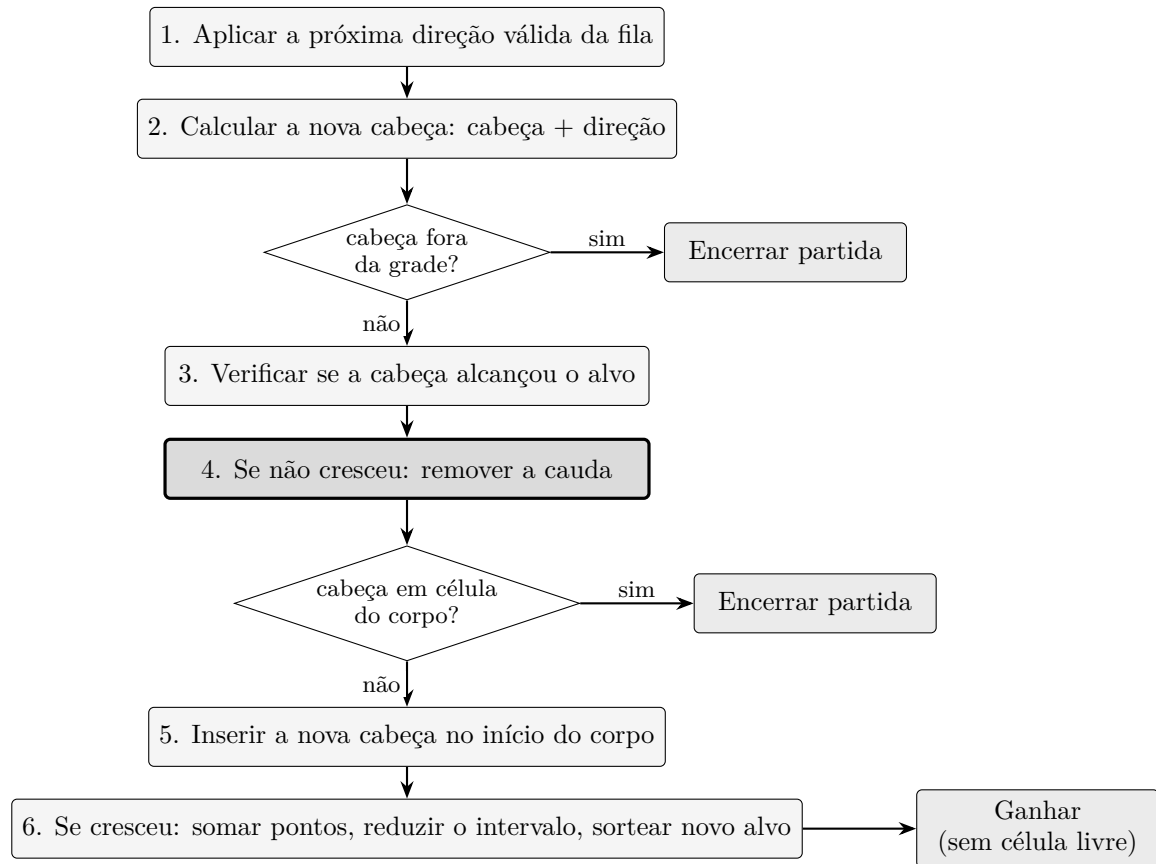
O passo de simulação é a unidade atômica do jogo. Sua ordem de operações é determinada por ADR-004 e está representada na Figura 9.

O Quadro 7.2 apresenta o trecho correspondente.

A ordem tem três propriedades que importam.

A direção é aplicada antes do movimento. A fila é consumida no início do passo, de modo que a direção usada no cálculo corresponde exatamente à intenção vigente naquele passo.

Figura 9 – Algoritmo de um passo de simulação, com destaque para a ordem de ADR-004



Fonte: elaborado pelo autor (2026) a partir de `index.html`, seção 8. O passo 4 é destacado porque sua posição *antes* do teste de colisão (ADR-004) determina se ocupar a célula recém-liberada pela cauda é sobrevivência ou morte.

A colisão de borda é testada antes de qualquer mutação do corpo. Se a cabeça saiu da grade, o estado não foi alterado. Isso mantém a consistência do estado observável em caso de encerramento.

A cauda é removida antes do teste de colisão com o corpo. Essa é a decisão ADR-004 e produz o comportamento declarado no caso de borda 5: mover para a célula que a cauda acabou de liberar é sobrevivência, não morte. A ordem inversa produziria a morte.

7.5 Fila de entrada e bloqueio de inversão

O bloqueio de inversão instantânea (FR-005) é implementado por fila de capacidade limitada com validação contra a última direção de referência. A Figura 10 apresenta a sequência de tratamento de uma entrada.

O Quadro 7.3 apresenta o trecho.

Quadro 7.2 – Trecho do passo de simulação: cálculo da cabeça, colisão e remoção da cauda (excertado)

```
1 if (state.queue.length > 0) {
2   state.dir = state.queue.shift();
3 }
4 const head = {
5   x: state.snake[0].x + state.dir.x,
6   y: state.snake[0].y + state.dir.y
7 };
8 if (head.x < 0 || head.y < 0 ||
9     head.x >= CONFIG.cols || head.y >= CONFIG.rows) {
10  endGame('collision');
11  return;
12 }
13 const growing = state.food !== null && sameCell(head, state.food);
14 if (!growing) {
15   state.snake.pop();
16 }
```

Fonte: `index.html`, seção 8.

Quadro 7.3 – Trecho do enfileiramento de direção com bloqueio de inversão (excertado)

```
1 const reference = state.queue.length > 0
2   ? state.queue[state.queue.length - 1]
3   : state.dir;
4
5 if (sameCell(next, reference) || isOpposite(next, reference)) {
6   return;
7 }
8 if (state.queue.length >= CONFIG.maxQueue) {
9   state.queue.shift();
10 }
11 state.queue.push({ ...next });
```

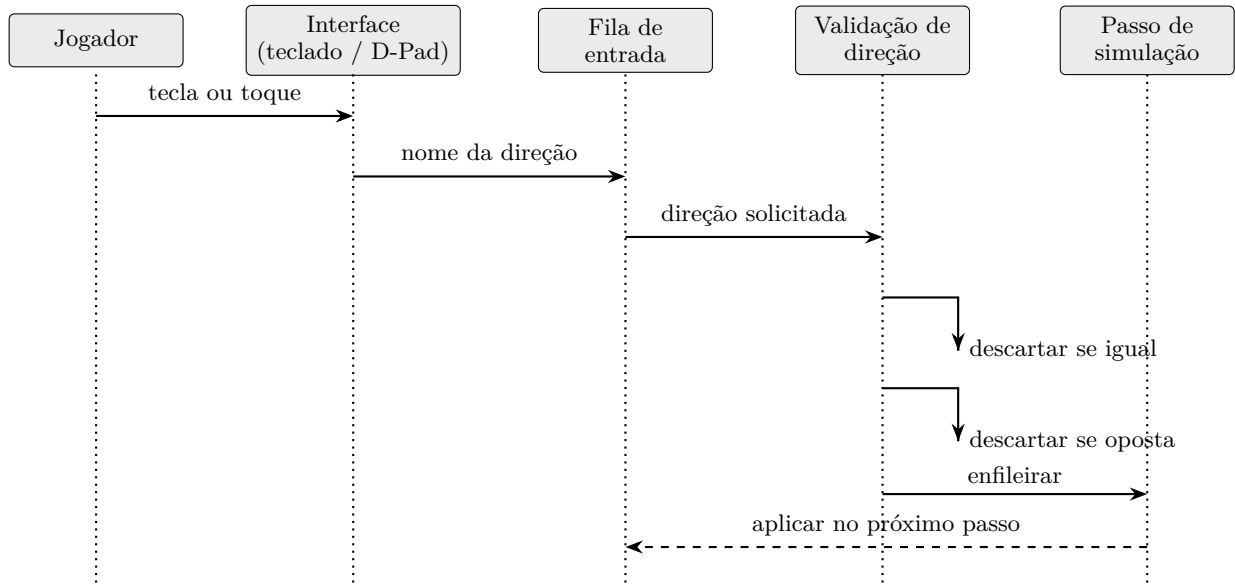
Fonte: `index.html`, seção 6.

A validação usa como referência a *última direção enfileirada*, e não a direção corrente, quando a fila não está vazia. A diferença é o que permite duas mudanças de noventa graus em um mesmo passo curto — por exemplo, direita seguida de baixo — sem permitir a inversão de cento e oitenta graus. Comparar apenas com a direção corrente rejeitaria a segunda entrada, e o jogador perderia o comando; comparar com a fila preserva a intenção sem abrir a possibilidade de inversão.

7.6 Sorteio do alvo

O sorteio do alvo (FR-006) impede que o alvo apareça sobre o corpo. A estratégia tem duas fases. Na primeira, sorteia-se uma célula aleatória e verifica-se sua presença

Figura 10 – Sequência de tratamento de uma entrada direcional



Referência de comparação: a última direção enfileirada, ou a direção corrente quando a fila está vazia.

Fonte: elaborado pelo autor (2026) a partir de `index.html`, seção 6 (ADR-003). Comparar com a última direção *enfileirada* permite duas mudanças de noventa graus no mesmo passo sem abrir a possibilidade de inversão de cento e oitenta graus.

em um conjunto de ocupação, por até 500 tentativas. Se todas falharem, executa-se uma varredura determinística em ordem de linha e coluna, retornando a primeira célula livre.

A escolha de duas fases é uma decisão de robustez. A amostragem aleatória é adequada enquanto a ocupação é baixa; quando o tabuleiro fica quase cheio, a probabilidade de acerto cai e o custo esperado de tentativas cresce. A varredura determinística garante término e, se nenhuma célula estiver livre, retorna valor nulo, que o passo de simulação interpreta como vitória — o estado `won`.

O Quadro 7.4 apresenta o núcleo do sorteio.

Quadro 7.4 – Trecho do sorteio de célula livre para o alvo (excerto)

```

1 const occupied = new Set(state.snake.map(function (cell) {
2   return cell.x + ',' + cell.y;
3 }));
4
5 for (let attempt = 0; attempt < CONFIG.maxFoodAttempts; attempt += 1) {
6   const cell = { x: randomInt(CONFIG.cols), y: randomInt(CONFIG.rows) };
7   if (!occupied.has(cell.x + ',' + cell.y)) {
8     return cell;
9   }
10 }
  
```

Fonte: `index.html`, seção 6.

Uma nota de eficiência: a codificação da célula como texto para uso em conjunto tem custo de alocação. A alternativa sem alocação seria uma matriz de ocupação reutilizada entre passos, o que exigiria limpeza seletiva a cada passo e introduziria risco de estado residual. A escolha pela legibilidade e previsibilidade, com o custo de alocação assumido, é coerente com NFR-005 e com o tamanho da grade (24 por 24 células).

7.7 Persistência do recorde

A persistência (FR-010) está isolada em duas funções envoltas em tratamento de exceção, conforme ADR-006. A gravação ocorre no momento em que a pontuação supera o recorde, e não apenas no encerramento da partida.

A escolha de gravar durante a partida tem uma consequência favorável: se a página for fechada abruptamente, o recorde conquistado não se perde. O custo é uma escrita no armazenamento por item consumido, custo irrelevante na escala do jogo. A leitura ocorre uma única vez, na inicialização.

O tratamento de exceção cobre dois cenários reais: navegação privada com armazenamento restrito e política de armazenamento desabilitada pelo usuário. Em ambos, a função de leitura retorna zero e o jogo permanece jogável com o recorde válido apenas na sessão corrente. A falha é degradada, não propagada.

7.8 Renderização

A renderização ocorre em superfície vetorial bidimensional (Mozilla Foundation (2024a)) e segue ADR-001. Três decisões a caracterizam.

Ajuste à densidade da tela. O Quadro 7.5 mostra o ajuste, que atende NFR-006. A razão de pixels é limitada a 2 para evitar custo de desenho desnecessário em telas de densidade muito alta, nas quais o ganho visual seria imperceptível e o custo de memória, não.

Conversão de célula para pixel em um único ponto. O tamanho da célula é calculado uma única vez, e todo o desenho deriva dele. Não há coordenada de pixel literal no código de desenho, exceto as relativas à própria célula (espessura de borda, deslocamento dos olhos). Essa disciplina é o que permite alterar o número de colunas da grade sem tocar no código de desenho.

Observação de redimensionamento. O ajuste é reexecutado quando o contêiner muda de tamanho, por observador de redimensionamento quando disponível e por evento de janela como alternativa. Isso cobre rotação de tela e redimensionamento de janela sem recarregar a página.

Quadro 7.5 – Trecho do ajuste da superfície de desenho à densidade da tela (excertado)

```
1 const rect = dom.canvas.getBoundingClientRect();
2 const cssSize = Math.max(1, Math.round(rect.width));
3 const dpr = Math.min(window.devicePixelRatio || 1, 2);
4
5 dom.canvas.width = Math.round(cssSize * dpr);
6 dom.canvas.height = Math.round(cssSize * dpr);
7 ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
8
9 view.size = cssSize;
10 view.cell = cssSize / CONFIG.cols;
```

Fonte: `index.html`, seção 9.

A Figura 11 registra visualmente o resultado da implementação no estado de fim de jogo: o tema escuro com acentos neon (NFR-003), o layout centralizado e responsivo (NFR-002), o placar com as duas grandezas (FR-009 e FR-010), a sobreposição de fim de jogo com o botão de reinício (FR-011) e os quatro controles direcionais (FR-004).

Cabe uma advertência sobre o estatuto dessa imagem. Ela é *registro visual*, não verificação. Nenhum critério de aceitação é satisfeito ou reprovado com base em uma captura de tela: os requisitos que ela ilustra são verificados por observação de comportamento (Tabela 6) ou, quando não possuem critério, permanecem sem verificação. Ilustrar um requisito em uma figura não o promove a verificado.

7.9 Acessibilidade e resposta a preferências

Três medidas atendem NFR-009 e às diretrizes de acessibilidade para conteúdo web (World Wide Web Consortium (2023)).

Preferência de movimento reduzido. A animação pulsante do alvo é desativada e as transições de interface são reduzidas quando o sistema operacional declara essa preferência. A escolha é relevante porque a pulsação é o único elemento puramente decorativo com movimento contínuo no produto.

Foco visível. Os elementos interativos têm indicação de foco explícita, o que permite operá-los por teclado. Sem isso, os botões direcionais seriam inacessíveis a quem não usa apontador.

Região viva para leitores de tela. Mudanças de estado da partida são anunciadas em região com semântica de estado, de modo que a transição para pausa ou para fim de jogo não seja silenciosa para tecnologias assistivas.

Registra-se uma limitação: os controles direcionais são botões com rótulo textual, o que os torna acessíveis, mas a superfície de desenho do jogo não oferece descrição textual

do estado corrente para leitores de tela. Um jogo acessível a pessoas cegas exigiria uma representação textual do tabuleiro, que não foi especificada e está fora do escopo deste trabalho.

7.10 Verificação de restrições durante a implementação

A Tabela 5 registra como cada restrição foi verificada na implementação.

Tabela 5 – Verificação das restrições na implementação

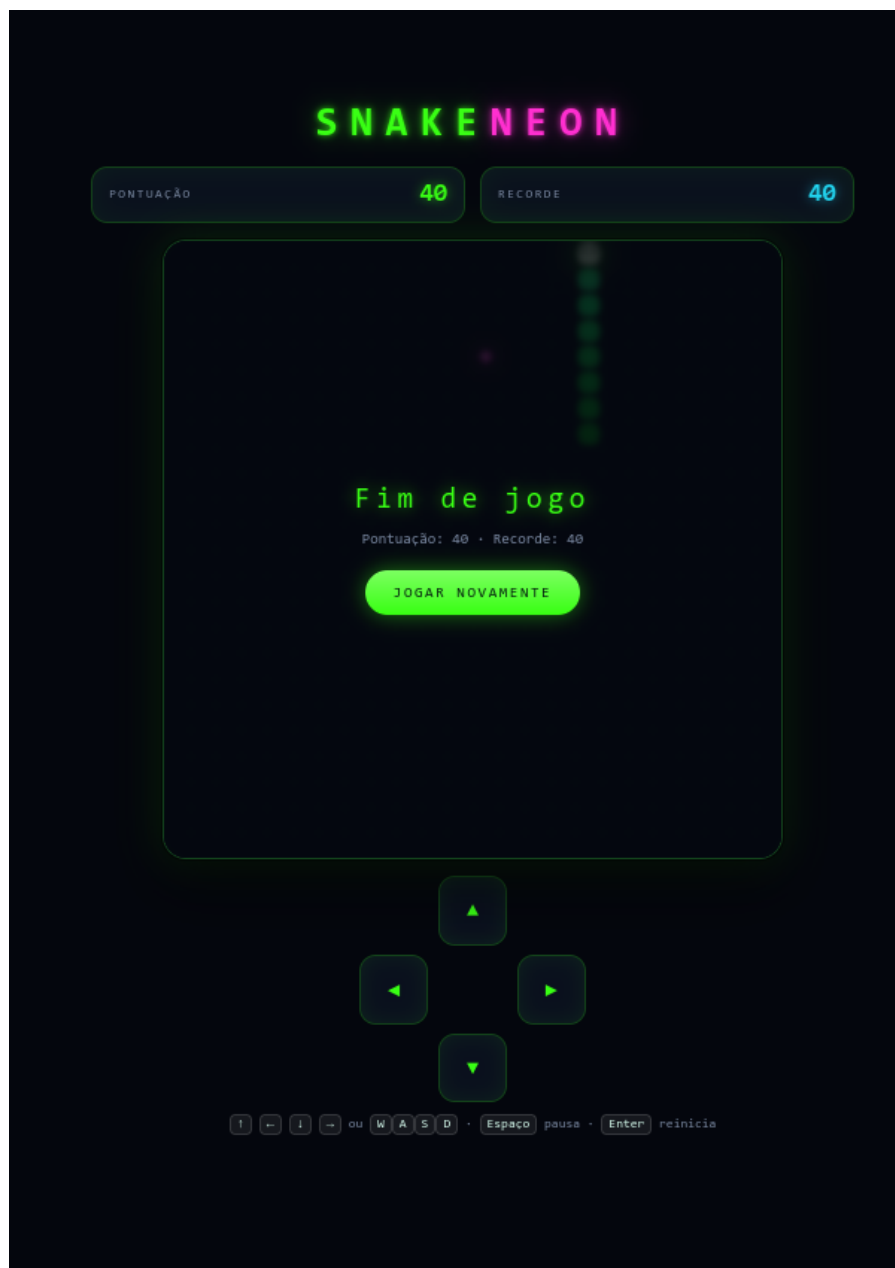
ID	Restrição	Verificação
CON-001	Artefato único	Inspeção: um único arquivo, um bloco de estilo, um bloco de script
CON-002	Apenas CSS e JavaScript nativos	Inspeção: ausência de referências de rede e de dependências
CON-003	Configuração centralizada	Inspeção: objeto de configuração único, congelado, com sete campos
CON-004	Sem temporizador fixo para simulação	Inspeção: laço por requisição de quadro com acumulador

Fonte: elaborado pelo autor (2026). A especificação do alvo declara quatro restrições (CON-001 a CON-004); as restrições de hardware do material de origem do framework não têm correspondência no produto (ver P-26).

7.11 Síntese do capítulo

A implementação materializou as sete decisões arquiteturais em 1 063 linhas organizadas em treze seções. Os pontos de maior consequência para o comportamento observável são a ordem entre remoção da cauda e teste de colisão (ADR-004), a referência usada na validação da fila de entrada (ADR-003) e o tratamento do estado inicial (FR-012). O Capítulo 8 verifica se o comportamento resultante corresponde ao especificado.

Figura 11 – Interface implementada, no estado de fim de jogo com placar preenchido



Fonte: arquivo `images/screenshot_gaming.png`. Captura de tela do sistema-alvo em execução, no estado `over`. São visíveis: o título da aplicação; o placar com pontuação atual e melhor pontuação, ambas em 40; o tabuleiro com o corpo do jogador; a sobreposição de fim de jogo com a pontuação final e o botão de reinício; os quatro botões direcionais; e a linha de atalhos de teclado. Esta figura é *registro visual* da interface, não verificação de critério de aceitação — a verificação está no Capítulo 8.

8 EXPERIMENTOS E RESULTADOS

Este capítulo reporta o protocolo de verificação e os resultados obtidos. Reportam-se apenas resultados efetivamente observados; o que não foi verificado está enumerado na Seção 8.7.

8.1 Protocolo de verificação

A verificação executou os doze critérios de aceitação do Quadro 6.4, mais uma medição quantitativa do intervalo entre passos de simulação. Para cada critério, o resultado foi classificado como satisfeito, não satisfeito ou não verificável. A classificação não admitiu valor intermediário.

Dois procedimentos foram usados. A inspeção estática verificou propriedades estruturais que não dependem de execução, por busca textual no artefato. A execução observada verificou comportamento, por automação de navegador com envio de eventos e leitura do estado observável da página, incluindo amostragem de pixels da superfície de desenho.

A opção por observação externa, sem instrumentar o produto, é deliberada. Um teste que exige alterar o código para expor o estado interno mede o código alterado, não o código entregue. A leitura de pixels contorna essa objeção: ela observa exatamente o que o usuário vê.

8.2 Resultados por critério de aceitação

A Tabela 6 consolida os resultados. Cada linha registra o critério, o instrumento usado, a observação e o resultado.

Substituindo na Equação (2.2), obtém-se

$$A = \frac{12}{12} = 1,00, \quad (8.1)$$

isto é, todos os critérios de aceitação declarados foram satisfeitos. O alcance dessa afirmação está limitado ao que está escrito na Seção 8.7.

Tabela 6 – Resultado da verificação por critério de aceitação

CA	Instrumento	Observação registrada	Resultado
CA-001	Inspeção táctica	Busca por referências de rede, de script e de estilo retornou vazio; exatamente um bloco de estilo e um de script	Satisfeito
CA-002	Execução servada	Entrada “direita” seguida de “esquerda”: o jogador manteve o sentido e encerrou por colisão com a borda direita	Satisfeito
CA-003	Execução servada	Agente de teste orientado por busca em largura: pontuação passou de 0 para 10 após consumo	Satisfeito
CA-004	Execução servada	25 amostras da célula do alvo; nenhuma sobreposição com células de corpo	Satisfeito
CA-005	Execução servada	Colisão com a borda: título da sobreposição passou a “Fim de jogo” e botão de reinício exibido	Satisfeito
CA-006	Execução servada	Após crescimento, giro fechado em quatro células encerrou a partida com a cabeça em célula interior (6,6), longe das bordas	Satisfeito
CA-007	Execução servada	Chave de armazenamento local com valor “10” e, em execução posterior, “100”; valor mantido após recarregar	Satisfeito
CA-008	Execução servada	Acionamento do botão superior moveu a cabeça de (11,11) para (12,9)	Satisfeito
CA-009	Execução servada	Sobreposição visível antes; oculta após a primeira direção válida	Satisfeito
CA-010	Execução servada	Estado “Pausado”: cabeça imóvel em (12,9) por 600 ms; retomada ocultou a sobreposição	Satisfeito
CA-011	Execução servada	Registro de erros e exceções do console vazio ao longo de todas as execuções	Satisfeito
CA-012	Medição	Intervalo médio 133 ms com pontuação abaixo de 40 e 105 ms com pontuação igual ou superior a 80 (Seção 8.5)	Satisfeito

Fonte: elaborado pelo autor (2026) a partir de `docs/05-testing/snake-game-evidence.md`.

8.3 Cobertura de rastreabilidade

Além do resultado por critério, mediu-se a cobertura de rastreabilidade: a fração de requisitos que possui critério de aceitação associado. A Tabela 7 apresenta o resultado, obtido por contagem direta na especificação.

Tabela 7 – Cobertura de rastreabilidade por classe de requisito

Classe	Total	Com CA	Parcial	Cobertura
Requisitos funcionais	15	11	1	73,3 %
Requisitos não funcionais	9	2	0	22,2 %
Total	24	13	1	54,2 %

Fonte: elaborado pelo autor (2026), por contagem em `spec.md`. “Parcial” indica requisito coberto apenas em parte pelo critério associado.

Os requisitos funcionais sem critério dedicado são FR-002 (uso da superfície de desenho), FR-009 (exibição do placar) e FR-014 (laço de animação). FR-011 está par-

cialmente coberto: o critério CA-005 verifica a exibição da tela de fim de jogo, mas não o funcionamento do botão de reinício, que foi observado empiricamente sem critério correspondente.

A assimetria entre as duas classes é o achado mais informativo desta seção. A cobertura de requisitos funcionais é de 73,3 %, enquanto a de requisitos não funcionais é de 22,2 %. A diferença não é acidental: requisitos funcionais descrevem comportamento observável e convertem-se naturalmente em teste; requisitos não funcionais, quando enunciados como atributos, não geram teste óbvio — e, sem teste, não geram critério. A consequência é que sete dos nove requisitos não funcionais ficaram fora de qualquer portão de aceitação.

Esse resultado não decorre do framework, e sim da especificação que o framework aceitou como válida. O portão G1 exige que existam requisito e critério antes da implementação; ele não exige que *todo* requisito tenha critério. Essa é uma lacuna do portão, identificada por autocrítica e discutida na Seção 9.2.1.

8.4 Verificação de propriedades estruturais

A inspeção estática produziu as seguintes observações concretas:

- um único arquivo, com exatamente um bloco de estilo e um bloco de script;
- nenhuma referência de rede a script, estilo, fonte ou imagem em qualquer ponto do artefato;
- sete campos no objeto de configuração, com valores idênticos aos especificados na Tabela 3;
- ausência de temporizador de intervalo fixo para a simulação, conforme CON-004;
- treze seções comentadas, conforme NFR-005.

Essas observações sustentam CA-001 e as restrições CON-001 a CON-004. Elas são o tipo de evidência mais barato de produzir e o mais barato de reproduzir, porque não depende de ambiente de execução.

8.5 Medição do intervalo entre passos

A única medição quantitativa de comportamento contínuo foi o intervalo entre passos de simulação, previsto pela Equação (6.2) e exigido por FR-015 e CA-012.

8.5.1 Procedimento

Um agente de teste jogou a partida de forma autônoma, localizando o alvo e a cabeça por amostragem de pixels e escolhendo o caminho mais curto por busca em largura sobre a grade livre. A cada mudança de posição da cabeça, registrou-se um par (instante, pontuação corrente). A partida foi conduzida até a pontuação de 100 pontos, sem encerramento por colisão.

Foram amostrados 184 movimentos. Os intervalos foram agrupados em duas faixas de pontuação e a média foi calculada pela Equação (4.1).

8.5.2 Resultado

A Tabela 8 confronta o valor medido com o intervalo previsto pelo modelo da Equação (6.2) para a faixa correspondente.

Tabela 8 – Intervalo médio entre passos medido e previsto, por faixa de pontuação

Faixa de pontuação	Média medida	Previsto (mín.)	Previsto (máx.)	Coerente
Abaixo de 40 pontos	133 ms	128 ms	140 ms	Sim
80 pontos ou mais	105 ms	100 ms	108 ms	Sim

Fonte: elaborado pelo autor (2026). Previsto pela Equação (6.2) nos extremos de cada faixa. Número de amostras por faixa não retido no registro de evidência — ver P-11.

Os dois valores medidos caem dentro do intervalo previsto para a faixa correspondente, com desvio de 1 ms em relação ao ponto médio de cada intervalo previsto. A diferença entre as duas faixas é de 28 ms, contra 28 ms previstos na comparação dos pontos médios (134 ms e 104 ms). A direção da variação — intervalo menor com pontuação maior — coincide com a exigida por FR-015.

Três advertências sobre a interpretação acompanham esse resultado.

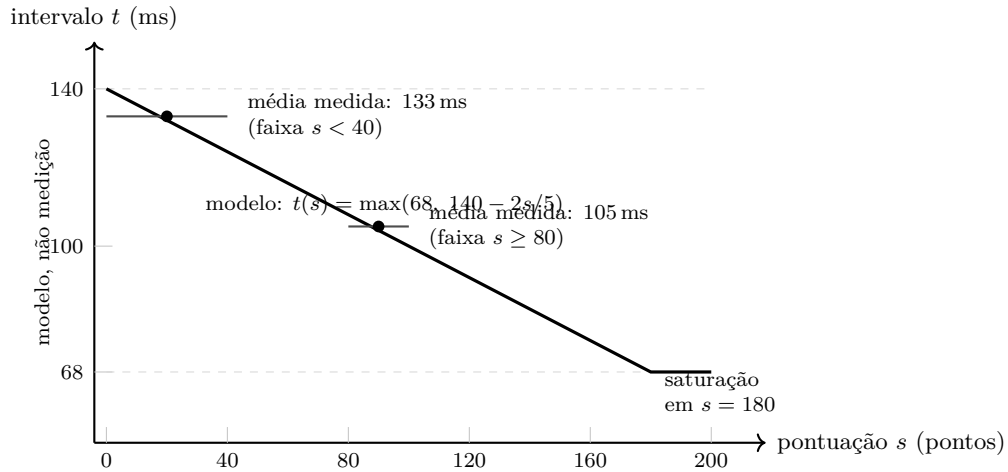
A medição é um limite superior. O ciclo de observação introduz atraso positivo: o instante registrado é posterior ao instante real da mudança de posição. Os valores medidos não devem ser lidos como estimativas do parâmetro `stepMs`, e sim como valores comparáveis entre si.

A média de faixa pressupõe distribuição aproximadamente uniforme da pontuação dentro da faixa. Essa suposição não foi verificada. Ela é plausível para a faixa alta, que cobre dois itens consumidos, mas não foi controlada.

A amostra é única. Não houve repetição do experimento, e portanto não há intervalo de confiança nem teste de hipótese. A conclusão sustentada é de coerência entre o medido e o previsto, não de significância estatística.

A Figura 12 representa a curva prevista pela especificação e posiciona os dois valores medidos. A curva é *modelo*, não medição; os dois pontos são *medição*, não modelo. A distinção é mantida visualmente e na legenda.

Figura 12 – Curva do modelo de velocidade da especificação e os dois valores medidos



Fonte: elaborado pelo autor (2026). A **curva** é o modelo declarado na especificação (Equação (6.2)), não uma medição. As **barras horizontais** são as faixas de pontuação sobre as quais cada média foi calculada, e os **círculos** são as médias medidas na Seção 8.5. Os valores medidos incluem atraso positivo do ciclo de observação e não devem ser lidos como estimativas do parâmetro **stepMs**.

8.6 Comportamentos observados sem critério correspondente

Durante a verificação, quatro comportamentos foram observados mas não possuem critério de aceitação associado. Registrá-los é obrigatório do ponto de vista da integridade: são resultados, ainda que não estivessem previstos como testes.

1. O reinício por tecla de confirmação na tela de fim de jogo restaura o estado e inicia a movimentação.
2. O reinício por tecla R restaura a partida a partir de qualquer estado ativo.
3. A perda de foco da janela provoca pausa automática.
4. Girando em quadrado com comprimento quatro, o jogador sobrevive; a partir de comprimento cinco, colide consigo mesmo. Esse comportamento é a consequência observável de ADR-004 e confirma materialmente o caso de borda 5.

O item 4 é o de maior valor para o trabalho, porque converte uma decisão arquitetural em comportamento observável e, portanto, em evidência. Ele mostra que ADR-004

não é uma preferência de estilo: produz diferença verificável no comportamento do produto para a mesma especificação de requisitos.

8.7 O que não foi verificado

A Tabela 9 enumera o que ficou fora da verificação. Nenhum item foi preenchido por estimativa.

Tabela 9 – Itens não verificados e razão

Item	Razão
Controles de toque em dispositivo móvel físico	Verificado apenas por emulação de evento de apontador em navegador de desktop. A diretriz do framework estabelece que emulação não promove o nível de evidência. Pendência registrada no projeto como T-016.
Consumo de memória e taxa de quadros	Não instrumentado. Nenhuma meta foi declarada na especificação, e portanto nenhum critério existe.
Responsividade em todas as larguras de 320 px a 1920 px	Verificada por cálculo de caixa de layout em uma largura e por inspeção dos pontos de quebra; não amostrada em todas as larguras.
Leitores de tela e tecnologias assistivas	A estrutura semântica foi inspecionada; não houve teste com leitor de tela real.
Navegadores alternativos ao motor Chromium	Não testado. A especificação não declara matriz de compatibilidade.
Comportamento sob armazenamento local indisponível	O tratamento de exceção existe por inspeção, mas o cenário não foi reproduzido em execução.

Fonte: elaborado pelo autor (2026).

8.8 Síntese do capítulo

A verificação produziu três resultados: doze de doze critérios de aceitação satisfeitos, uma cobertura de rastreabilidade de 54,2 % sobre o conjunto de requisitos, com forte assimetria entre requisitos funcionais (73,3 %) e não funcionais (22,2 %), e uma medição de intervalo entre passos coerente com o modelo da especificação em duas faixas de pontuação, com diferença de 28 ms entre elas. Seis itens permaneceram fora da verificação, e nenhum foi preenchido por estimativa.

9 DISCUSSÃO

Este capítulo interpreta os resultados, confronta-os com o problema e a hipótese, identifica as lacunas que a própria verificação revelou e retoma as ameaças à validade. A seção final discute em que medida os achados são generalizáveis.

9.1 Retomada do problema e da hipótese

O problema enunciado na Seção 1.2 tinha três componentes: rastreabilidade a requisito, verificabilidade por critério e sustentação por evidência reproduzível. Os três foram examinados, com resultados desiguais.

A **verificabilidade por critério** foi o componente mais bem-sucedido. Os doze critérios declarados foram satisfeitos, e cada um foi verificado por observação externa, sem consultar o autor e sem instrumentar o produto. A consequência relevante é metodológica: como os critérios foram redigidos de forma que um observador externo possa decidir o resultado, verificar deixou de exigir acesso ao raciocínio de quem implementou.

A **sustentação por evidência reproduzível** também se confirmou. A inspeção estática é reproduzível por comando; a execução observada é reproduzível por automação; a medição de intervalo é reproduzível por agente de teste que observa a superfície de desenho. Nenhuma conclusão deste trabalho depende de lembrança ou de registro narrativo.

A **rastreabilidade a requisito** foi o componente que falhou parcialmente. A cobertura de 54,2% significa que quase metade dos requisitos não possui critério de aceitação associado. A cadeia requisito–critério–evidência existe, mas apenas para parte do conjunto especificado. A hipótese, portanto, é *parcialmente* sustentada: o processo produz entregas verificáveis, mas não garante que a verificabilidade cubra a especificação inteira.

9.2 A assimetria entre requisitos funcionais e não funcionais

O achado mais relevante do trabalho é a diferença de cobertura entre as classes de requisito: 73,3% para funcionais e 22,2% para não funcionais. A magnitude da diferença descarta a explicação por acaso.

A explicação é estrutural. Um requisito funcional descreve um comportamento observável, e comportamento observável tem forma de teste quase natural: dado um estado, ao executar uma ação, espera-se uma consequência. Requisito não funcional enunciado como atributo — “layout moderno e limpo”, “tema escuro de estilo retro-arcade” — não

descreve consequência observável; descreve uma característica percebida. Sem um passo intermediário de conversão em métrica, ele não gera critério, e sem critério não entra em portão algum.

Esse passo intermediário não é trivial e não foi executado. A Tabela 1 mostra o sintoma: três dos nove requisitos não funcionais recaem em características de qualidade para as quais nenhuma métrica foi declarada. O requisito existe, a característica de qualidade foi identificada, mas o elo que os liga a um teste não foi construído.

A conclusão prática é que a exigência de critério de aceitação deve ser *universal*, não apenas aplicada aos requisitos que naturalmente se convertem em teste. Um requisito sem critério é um requisito não verificável, e um requisito não verificável é indistinguível de uma intenção. O framework precisa de um portão que rejeite requisito órfão de critério, ou que force sua conversão em métrica ou seu rebaixamento explícito a diretriz não verificável.

9.2.1 A lacuna do portão de intenção

O portão G1 exige que existam um requisito identificado e um critério de aceitação antes do avanço para a implementação. A redação do portão, entretanto, é satisfeita pela existência de *ao menos* um par requisito–critério. Ela não impõe que todo requisito declarado possua critério correspondente.

A consequência é assimétrica e foi medida: o portão deixa passar uma especificação em que onze requisitos funcionais e dois não funcionais têm critério, e dez requisitos não têm nenhum. A verificação subsequente de doze critérios produz a impressão de cobertura completa, porque todos os critérios declarados foram satisfeitos. A lacuna permanece invisível até que alguém confronte explicitamente a contagem de requisitos com a contagem de critérios — procedimento que, no caso deste trabalho, só foi executado na etapa de consolidação da matriz de rastreabilidade.

Duas correções são possíveis, e elas não são equivalentes. A primeira é tornar o portão mais estrito: bloquear o avanço enquanto houver requisito sem critério. A segunda é tornar o portão mais informativo: exigir que todo requisito sem critério seja explicitamente rebaixado a “diretriz não verificável neste ciclo”. A segunda é preferível, porque existem requisitos legitimamente não verificáveis por teste automatizado — aparência visual, por exemplo — e bloquear a implementação por causa deles tornaria o processo inaplicável. O rebaixamento explícito resolve o problema real, que não é a ausência de teste, e sim a ausência de *consciência* da ausência de teste.

Formalizando: seja R o conjunto de requisitos e $R_c \subseteq R$ o subconjunto com critério de aceitação dedicado. O portão atual exige $|R_c| \geq 1$. A correção proposta exige que $R \setminus R_c$ esteja declarado como conjunto de diretrizes não verificáveis neste ciclo, e não que $R \setminus R_c = \emptyset$. Neste estudo de caso, $|R| = 24$ e $|R_c| = 13$, de modo que $|R \setminus R_c| = 11$: dez

requisitos sem critério algum e um coberto apenas parcialmente. Esse é o conjunto que hoje não está declarado como não verificável.

9.3 Valor dos casos de borda e das decisões arquiteturais

Dois resultados empíricos sustentam a tese de que declarar casos de borda e registrar decisões não é cerimônia documental.

O primeiro é o estado de vitória. O caso de borda “grade completamente ocupada” foi declarado na especificação antes da implementação. Ao implementar o sorteio do alvo, tornou-se evidente que essa situação exigia um estado de jogo que não existia na modelagem inicial. O caso de borda *criou* um estado (Figura 7). Sem ele, o sorteio retornaria valor inválido em uma situação alcançável, e o defeito só apareceria quando alguém completasse o tabuleiro — provavelmente nunca, em uso normal.

O segundo é o comportamento da cauda. O caso de borda 5 e a decisão ADR-004 combinam-se para produzir um comportamento observável e não intuitivo: girando em quadrado com comprimento quatro o jogador sobrevive; com comprimento cinco, colide consigo mesmo (Seção 8.6, item 4). Dois implementadores igualmente competentes, partindo da mesma especificação de requisitos e sem o registro de ADR-004, produziriam comportamentos diferentes. A decisão arquitetural é o que torna o resultado determinístico. Isso responde ao argumento de que registrar decisões é documentação supérflua: nesse caso, o registro elimina uma ambiguidade real de implementação.

9.4 Custo do processo

O processo tem custo, e ele deve ser declarado. A disciplina exigida produziu cinco documentos de especificação e projeto antes da primeira linha de código: especificação, decisões arquiteturais, tarefas, evidências e controle de estado. Para um produto de 1 063 linhas, essa proporção é alta.

Três observações qualificam o custo.

O custo é antecipado, não adicionado. A declaração prévia de casos de borda evitou defeitos que seriam descobertos tardiamente, quando o custo de correção é maior (Equação (2.1)). O estado de vitória é o exemplo concreto: descobri-lo em produção custaria mais do que especificá-lo.

O custo é amortizado na verificação. Critérios redigidos antes da implementação produziram verificação executável por terceiros. Critérios redigidos depois teriam exigido reinterpretação do autor a cada verificação.

O custo é proporcional ao risco, não ao tamanho. O fluxo adaptativo (Seção 5.7) existe precisamente para evitar aplicar a cadeia completa a alterações pequenas. Neste estudo de caso, a trilha aplicada foi a de recurso médio com elevação a amplo, decisão coerente com o fato de o alvo ter sido especificado do zero.

Não foi possível quantificar o custo em tempo ou em tokens, porque a execução não foi instrumentada (P-12, P-13). A avaliação acima é qualitativa e não substitui a medição ausente.

9.5 Ameaças à validade revisitadas

As sete ameaças do Quadro 4.4 são retomadas aqui quanto ao efeito sobre as conclusões.

AV-1, ausência de grupo de controle, é a ameaça mais séria. Sem um grupo que executasse o mesmo alvo sem o processo, não é possível atribuir os doze critérios satisfeitos ao processo. Uma explicação alternativa é que o alvo é simples e que um implementador competente obteria o mesmo resultado sem especificação formal. O trabalho não refuta essa alternativa. O que ele demonstra é mais modesto e ainda assim relevante: que o processo torna o resultado *verificável por terceiros sem acesso ao histórico*, propriedade que não foi medida no grupo alternativo justamente porque esse grupo não existiu.

AV-6, amostra única, limita a medição de velocidade. O intervalo medido é coerente com o previsto, mas coerência em uma execução não é confirmação estatística. Um resultado mais forte exigiria repetição e intervalo de confiança.

AV-3, autoria coincidente, é mitigada pela natureza da evidência. A verificação foi produzida por execução observada e inspeção textual, não por julgamento do autor. Um terceiro pode repetir os comandos e obter os mesmos resultados. Isso não elimina o risco de confirmação na escolha dos critérios, mas remove o risco de confirmação na produção da evidência.

AV-4, a métrica de rastreabilidade mede existência e não qualidade. Um critério mal redigido contaria como cobertura. A ameaça é real e reduz o valor do número de 54,2%. Ela não afeta, entretanto, o achado principal, que é a *assimetria* entre as classes: se critérios ruins inflassem artificialmente a cobertura, a cobertura de requisitos não funcionais seria ainda menor, e a assimetria, mais acentuada.

AV-2 e AV-5 permanecem nos limites declarados: generalização restrita a domínios de complexidade comparável e viés positivo nos valores absolutos de intervalo.

9.6 Generalização

A generalização de um estudo de caso único é analítica, não estatística (Yin (2018)): o resultado não se estende a uma população, mas confirma ou refuta proposições teóricas. Três proposições recebem suporte limitado dos dados.

Proposição 1. Requisitos funcionais convertem-se em critério de aceitação com mais facilidade que requisitos não funcionais. Sustentada por evidência direta neste caso (73,3 % contra 22,2 %). A proposição é plausível em qualquer domínio de software; a magnitude específica não é transferível.

Proposição 2. Declarar casos de borda antes da implementação altera o projeto, não apenas os testes. Sustentada por evidência direta (o estado de vitória). A proposição decorre diretamente dos dados e não depende do domínio.

Proposição 3. Evidência produzida por observação externa ao produto é reproduzível por terceiros sem instrumentar o código. Sustentada pela construção metodológica deste trabalho, não por comparação.

O que não é transferível. O domínio do estudo de caso tem estado discreto, regras determinísticas e ausência de concorrência e de integração externa. Em domínios com estado contínuo, tempo real estrito, sistema distribuído ou dependência de hardware, a conversão de requisitos em critério é substancialmente mais difícil, e o processo exigiria níveis de evidência que este trabalho não alcança. O próprio material de origem do framework previa quatro níveis de evidência — teste em máquina hospedeira, simulação, bancada e integração com hardware — e este estudo permaneceu no nível mais baixo. A aplicação a domínios de nível mais alto permanece como questão aberta.

9.7 Comparação com a literatura

Três convergências merecem registro.

A ênfase de Boehm (1984) na verificação precoce de requisitos e projeto encontra apoio no achado da Seção 9.3: a declaração prévia de casos de borda alterou a modelagem de estados antes que houvesse código.

A tese de Adzic (2011) de que o exemplo concreto serve simultaneamente como requisito, teste e documentação é confirmada pelo formato dos critérios usados neste trabalho, redigidos em forma de situação, ação e consequência esperada.

A observação de Myers, Sandler e Badgett (2011) de que um caso de teste que não pode falhar não tem valor é o critério pelo qual se identifica a lacuna dos requisitos não funcionais: os sete requisitos sem critério não possuem caso de teste capaz de reprová-los, e por isso não entraram em nenhum portão.

Em relação a Hong et al. (2024), Qian et al. (2024) e Wu et al. (2023), a diferença de ênfase apontada no Capítulo 3 é confirmada por este estudo: aqueles trabalhos maximizam a qualidade do artefato gerado pela coordenação de papéis; este trabalho deslocou o foco para os artefatos que tornam a geração auditável. Os dois enfoques são complementares, não concorrentes.

9.8 Implicações práticas

Quatro implicações decorrem dos resultados.

Para o processo. A exigência de critério deve ser universal e verificada por portão. Requisito sem critério deve ser bloqueado ou explicitamente rebaixado a diretriz não verificável.

Para a especificação. Requisito não funcional deve ser redigido já com sua métrica, ou com a declaração explícita de que não é verificável neste ciclo. A declaração explícita é preferível à omissão, porque torna a lacuna visível.

Para a verificação. Observar o produto por fora, sem instrumentá-lo, resolve o conflito entre testar e alterar o objeto de teste. Em interfaces gráficas discretas, a amostragem da superfície de desenho é uma técnica viável e de baixo custo.

Para a condução por agentes. As condições de parada cumpriram função real: a divergência entre a documentação do repositório e seu conteúdo efetivo foi detectada e registrada em vez de resolvida por suposição silenciosa. Sem uma condição de parada explícita, a resposta mais provável teria sido produzir documentação sobre um sistema inexistente.

10 CONCLUSÃO

10.1 Resposta ao problema

O problema investigado foi como estruturar um processo de desenvolvimento assistido por agentes de inteligência artificial de modo que cada entrega fosse rastreável a um requisito, verificável por um critério de aceitação e sustentada por evidência reproduzível.

O trabalho responde em três partes.

Quanto à verificabilidade, a resposta é afirmativa. Um processo que obriga a redigir critérios de aceitação antes da implementação, em forma que um observador externo possa decidir o resultado, produz entregas verificáveis por terceiros sem acesso ao raciocínio de quem implementou. Os doze critérios declarados foram satisfeitos e verificados por observação externa ao produto.

Quanto à sustentação por evidência, a resposta é afirmativa. A separação entre o que o agente afirma e o que a execução demonstra é o mecanismo central do framework. Toda conclusão deste trabalho se apoia em inspeção textual reproduzível, em execução observada ou em medição; nenhuma se apoia em afirmação do executor.

Quanto à rastreabilidade, a resposta é parcial. A cadeia requisito–critério–evidência existe e é verificável, mas cobre 54,2% do conjunto de requisitos. A cobertura é de 73,3% para requisitos funcionais e de apenas 22,2% para não funcionais. O processo entrega rastreabilidade onde ela é exigida com rigor, mas não garante cobertura completa, e a lacuna se concentra exatamente na classe de requisito cuja verificação é mais difícil.

10.2 Cumprimento dos objetivos

O Quadro 10.1 retoma os objetivos específicos e registra o grau de cumprimento.

O objetivo específico OE-6 é classificado como parcial porque a avaliação identificou ameaças que não puderam ser mitigadas dentro do escopo, em particular a ausência de grupo de controle e a ausência de repetição da medição. Identificar e declarar uma limitação é resultado válido, mas não é o mesmo que eliminá-la.

Quadro 10.1 – Grau de cumprimento dos objetivos específicos

ID	Resultado	Grau
OE-1	Framework definido com cinco camadas, quinze agentes, quatro conjuntos de instruções, cinco habilidades, seis portões, catorze itens de conclusão e seis condições de parada.	Pleno
OE-2	Especificação com 15 requisitos funcionais, 9 não funcionais, 4 restrições, 12 critérios de aceitação e 5 casos de borda.	Pleno
OE-3	Sete decisões arquiteturais registradas com justificativa e alternativa descartada.	Pleno
OE-4	Artefato único de 1 063 linhas, sem dependências externas, conforme as quatro restrições do alvo.	Pleno
OE-5	Doze de doze critérios satisfeitos, com evidência registrada por critério.	Pleno
OE-6	Sete ameaças à validade classificadas, seis dados ausentes marcados, lacuna de rastreabilidade identificada e localizada.	Parcial

Fonte: elaborado pelo autor (2026).

10.3 Contribuições

1. **Um protocolo de controle operacional** para desenvolvimento com agentes de inteligência artificial, com precedência do critério sobre a implementação e proibição de evidência fabricada, materializado em artefatos versionados e reutilizáveis.
2. **Um método de verificação por observação externa** que não instrumenta o produto sob teste, viável em interfaces gráficas discretas por amostragem da superfície de desenho e uso de agente de teste autônomo.
3. **Evidência empírica de uma assimetria** entre a verificabilidade de requisitos funcionais e não funcionais, com medição da cobertura por classe: 73,3 % contra 22,2 %.
4. **Demonstração de que casos de borda declarados alteram o projeto**, e não apenas os testes, com um caso concreto em que a declaração prévia criou um estado de máquina que não existia na modelagem inicial.
5. **Registro do custo e das limitações** do processo, incluindo a declaração explícita do que não foi verificado e de quais dados não foram obtidos, em vez de preenchimento por estimativa.

10.4 Limitações

As limitações deste trabalho são de quatro naturezas.

Metodológica. Não houve grupo de controle; o desenho é de estudo de caso único, com generalização analítica e não estatística. Não houve repetição da medição de intervalo, e portanto não há inferência estatística sobre ela.

De escopo. O alvo tem estado discreto, regras determinísticas e ausência de concorrência, integração externa e dependência de hardware. Os resultados não se transferem diretamente a domínios de tempo real, sistemas distribuídos ou software embarcado, nos quais o framework de origem previa níveis de evidência superiores.

De cobertura. Sete dos nove requisitos não funcionais não possuem critério de aceitação. Sete itens permaneceram fora da verificação, incluindo validação em dispositivo de toque físico, compatibilidade entre navegadores e comportamento sob armazenamento local indisponível.

De evidência. Nenhum dado de consumo de memória, de tempo de execução do processo ou de custo por tarefa foi coletado. As afirmações sobre custo do processo são qualitativas.

10.5 Trabalhos futuros

1. **Adicionar portão de requisito órfão.** Estender o portão G1 para rejeitar requisito sem critério de aceitação, ou obrigar seu rebaixamento explícito a diretriz não verificável. É a correção diretamente indicada pelo achado principal deste trabalho.
2. **Instrumentar o processo.** Medir tempo, número de iterações e custo de contexto por tarefa, convertendo a avaliação qualitativa de custo em avaliação quantitativa.
3. **Executar experimento controlado.** Desenvolver o mesmo alvo com e sem o processo, com métricas comparáveis, para atacar a ameaça AV-1.
4. **Repetir a medição de velocidade** com múltiplas execuções e tratamento estatístico, com registro do número de amostras por faixa.
5. **Aplicar a domínio de nível de evidência superior.** Verificar o comportamento do protocolo em desenvolvimento embarcado, com teste em bancada e integração com hardware, domínio para o qual o material de origem do framework foi concebido.
6. **Avaliar a conversão de requisitos não funcionais em métricas** como etapa assistida por agente, verificando se um agente dedicado reduz a assimetria medida.

10.6 Considerações finais

A contribuição central deste trabalho não é um produto de software nem um modelo de linguagem. É uma demonstração de que a disciplina de especificação, aplicada a um fluxo conduzido por agentes de inteligência artificial, produz um resultado auditável — e de que essa disciplina tem um ponto cego mensurável exatamente onde a verificação é mais difícil.

O achado de que a cobertura de requisitos não funcionais é de 22,2 % é, do ponto de vista da engenharia, mais útil que a afirmação de que doze critérios foram satisfeitos. O primeiro resultado localiza um defeito no processo e indica a correção; o segundo apenas confirma que o processo funcionou onde era fácil funcionar. Um processo de verificação que não expõe sua própria lacuna é um processo que ainda não foi verificado.

REFERÊNCIAS

- ADZIC, G. *Specification by Example: How Successful Teams Deliver the Right Software*. Shelter Island: Manning, 2011.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software Architecture in Practice*. 4. ed. Boston: Addison-Wesley, 2021.
- BECK, K. *Test-Driven Development: By Example*. Boston: Addison-Wesley, 2002.
- BOEHM, B. W. Verifying and validating software requirements and design specifications. *IEEE Software*, 1984.
- BROWN, T. B. et al. Language models are few-shot learners. In: *Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2020.
- Ecma International. *ECMA-262: ECMAScript Language Specification*. 2023. Norma técnica.
- FORSGREN, N.; HUMBLE, J.; KIM, G. *Accelerate: The Science of Lean Software and DevOps*. Portland: IT Revolution Press, 2018.
- GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1994.
- HONG, S. et al. Metagpt: Meta programming for a multi-agent collaborative framework. In: *International Conference on Learning Representations*. [S.l.: s.n.], 2024.
- HUMBLE, J.; FARLEY, D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston: Addison-Wesley, 2010.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva, 2011. Verificar a edição vigente antes da entrega.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *ISO/IEC/IEEE 29148: Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva, 2018. Verificar a edição vigente antes da entrega.
- MERNIK, M.; HEERING, J.; SLOANE, A. M. When and how to develop domain-specific languages. *ACM Computing Surveys*, 2005.
- Mozilla Foundation. *Canvas API — MDN Web Docs*. 2024. Documentação técnica oficial.
- Mozilla Foundation. *Window.localStorage — MDN Web Docs*. 2024. Documentação técnica oficial.
- Mozilla Foundation. *Window.requestAnimationFrame() — MDN Web Docs*. 2024. Documentação técnica oficial.

- MYERS, G. J.; SANDLER, C.; BADGETT, T. *The Art of Software Testing*. 3. ed. Hoboken: John Wiley & Sons, 2011.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.
- QIAN, C. et al. Chatdev: Communicative agents for software development. In: *Annual Meeting of the Association for Computational Linguistics*. [S.l.: s.n.], 2024.
- RUNESON, P.; HÖST, M. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 2009.
- RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. Hoboken: Pearson, 2021.
- SHINN, N. et al. Reflexion: Language agents with verbal reinforcement learning. In: *Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2023.
- SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.
- VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2017.
- Web Hypertext Application Technology Working Group. *HTML Living Standard*. 2024. Especificação técnica. Documento em constante atualização.
- WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. In: *Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2022.
- WOHLIN, C. et al. *Experimentation in Software Engineering*. Berlin: Springer, 2012.
- WOOLDRIDGE, M. *An Introduction to MultiAgent Systems*. 2. ed. Chichester: John Wiley & Sons, 2009.
- World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. 2023. W3C Recommendation.
- WU, Q. et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. ArXiv:2308.08155.
- YAO, S. et al. React: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations*. [S.l.: s.n.], 2023.
- YIN, R. K. *Case Study Research and Applications: Design and Methods*. 6. ed. Los Angeles: SAGE, 2018.

Apêndices

APÊNDICE A – MATRIZ DE RASTREABILIDADE COMPLETA

Este apêndice apresenta a matriz de rastreabilidade do sistema-alvo no formato de sete colunas definido no framework: requisito, critério, tarefa, código, teste, evidência e estado. A matriz é a fonte primária da Tabela 7 e da Tabela 6.

Consulte o Quadro 6.1, o Quadro 6.2 e o Quadro 6.4 para o texto integral de cada identificador.

Tabela 10 – Matriz de rastreabilidade requisito–critério–tarefa–código–teste–evidência

Requisito	Critério	Tarefa	Código	Teste	Evidência	Estado
FR-001	CA-001	T-001	estrutura	inspeção	E-01, E-11	PASS
FR-002	—	T-009	render	—	E-01	SEM CA
FR-003	CA-008	T-010	entrada	execução	E-18	PASS
FR-004	CA-008	T-011	entrada	execução	E-18	PASS
FR-005	CA-002	T-006	fila	execução	E-04, E-12	PASS
FR-006	CA-004	T-006	sorteio	execução	E-06, E-14	PASS
FR-007	CA-003	T-007	passo	execução	E-13	PASS
FR-008	CA-005, CA-006	T-007	passo	execução	E-05, E-15, E-16	PASS
FR-009	—	T-003	HUD	—	E-17	SEM CA
FR-010	CA-007	T-005	persistência	execução	E-07, E-17	PASS
FR-011	CA-005 (parcial)	T-012	sobreposição	execução	E-15	PARCIAL
FR-012	CA-009	T-012	estados	execução	E-19	PASS
FR-013	CA-010	T-012	estados	execução	E-20	PASS
FR-014	—	T-008	laço	—	E-03	SEM CA
FR-015	CA-012	T-007	passo	medição	E-10, E-22	PASS

continua na próxima página

Requisito	Critério	Tarefa	Código	Teste	Evidência	Estado
NFR-001	CA-001	T-001	estrutura	inspeção	E-02, E-11	PASS
NFR-002	—	T-002	estilo	—	E-01	SEM CA
NFR-003	—	T-002	estilo	—	E-01	SEM CA
NFR-004	—	T-002	estilo	—	E-08	SEM CA
NFR-005	—	T-013	todo	—	E-01	SEM CA
NFR-006	—	T-009	redimensionamento	—	E-08	SEM CA
NFR-007	—	T-010	entrada	—	E-04	SEM CA
NFR-008	CA-011	T-014	todo	execução	E-21	PASS
NFR-009	—	T-002	estilo	—	E-09	SEM CA

A.1 Leitura da matriz

A coluna **Estado** admite três valores. **PASS** indica critério verificado com evidência observada. **SEM CA** indica requisito sem critério de aceitação associado, e portanto sem possibilidade de verificação. **PARCIAL** indica requisito coberto apenas em parte pelo critério associado.

Dez requisitos aparecem como **SEM CA** — FR-002, FR-009, FR-014, NFR-002, NFR-003, NFR-004, NFR-005, NFR-006, NFR-007 e NFR-009 — e um aparece como **PARCIAL** (FR-011). Os treze requisitos restantes possuem critério dedicado. A contagem sobre o conjunto total de 24 requisitos produz a cobertura de 54,2% reportada na Seção 8.3.

A matriz é apresentada sem correção retroativa. Corrigir a especificação agora, acrescentando critérios aos requisitos descobertos, invalidaria o relato da Seção 8.3: o achado depende de a lacuna ter existido durante a execução. A correção proposta está na Seção 10.5.

APÊNDICE B – LISTA DE VERIFICAÇÃO DE ENCERRAMENTO APLICADA

Este apêndice aplica ao estudo de caso a lista de verificação de encerramento do framework, com catorze itens. Para cada item registra-se a situação observada e a evidência correspondente. A coluna “Aplicável” distingue os itens que não se aplicam ao tipo de artefato produzido — distinção necessária porque uma lista de verificação com itens permanentemente inaplicáveis deixa de ser um instrumento de controle.

Tabela 11 – Lista de verificação de encerramento aplicada ao estudo de caso

#	Item	Aplic.	Situação observada
1	Requisito relacionado identificado	Sim	Todo artefato aponta para identificador FR, NFR ou CA
2	Critérios de aceitação verificados	Sim	12 de 12 satisfeitos (Tabela 6)
3	Estado do repositório inspecionado antes da alteração	Não	O diretório não era repositório válido no início da execução. Item não cumprido, registrado no STATUS.md
4	Alteração no menor escopo necessário	Sim	Cada tarefa restrita ao módulo afetado; nenhuma refatoração não relacionada
5	Compilação do alvo executada	Não	Não aplicável: artefato interpretado diretamente pelo navegador, sem etapa de compilação
6	Testes executados	Sim	12 verificações por inspeção e execução observada
7	Análise estática executada quando aplicável	Sim	Busca textual de referências externas e de temporizadores fixos

#	Item	Aplic.	Situação observada
8	Consumo de memória e tempo verificados quando aplicável	Não	Não medido. Registrado como dado necessário P-10
9	Documentação atualizada	Sim	Documento de estado e documentação de uso produzidos
10	Rastreabilidade atualizada	Sim	Matriz do Apêndice A
11	Arquivo de estado atualizado	Sim	Documento de estado com pendências e riscos residuais
12	Arquivo de transferência atualizado apenas se houver continuidade	Não	Não houve transferência a outro responsável na execução relatada
13	Riscos residuais registrados	Sim	Quatro riscos residuais, incluindo a pendência de validação em dispositivo físico
14	Nenhuma alteração não relacionada incorporada	Sim	Escopo preservado; nenhuma dependência adicionada

B.1 Leitura da lista

Dos catorze itens, nove foram cumpridos, quatro não se aplicam ao tipo de artefato e um — a inspeção do estado do repositório antes da alteração — não foi cumprido, por indisponibilidade do próprio mecanismo de controle de versão no ambiente de execução.

A consequência prática do item 3 é relevante e merece registro. O portão G0 do framework exige a leitura do estado do repositório antes de qualquer alteração, precisamente para impedir que o executor trate uma versão lembrada de um arquivo como se fosse a versão presente. Como o repositório não era válido no início da execução, o portão não pôde ser cumprido, e a mitigação adotada foi a leitura direta dos arquivos antes de cada alteração.

Isso expõe uma fragilidade do framework: um portão que depende de infraestrutura externa não possui plano alternativo declarado. A correção proposta é que o portão G0 defina explicitamente o procedimento substituto quando o sistema de controle de versão não estiver disponível, em vez de simplesmente não ser cumprido.